算法模板汇总

目录

- 赛场配置 - 第 2 页：Ubuntu ICPCOS
- 动态规划 - 第 3 页：SOS DP
- 动态规划 - 第 3 页：01背包问题
- 动态规划 - 第 4 页：多重背包问题
- 动态规划 - 第 4 页：多重背包问题的二进制优化
- 动态规划 - 第 5 页：完全背包问题
- 图论 - 第 5 页：Kruskal 重构树
- 图论 - 第 7 页：最小生成树
- 图论 - 第 9 页：Bellman–Ford 算法
- 图论 - 第 10 页：Dijkstra 算法
- 图论 - 第 11 页：Floyd 算法
- 图论 - 第 12 页：Johnson 全源最短路径算法
- 图论 - 第 14 页：SPFA算法
- 图论 - 第 14 页：差分约束
- 图论 - 第 16 页：树上启发式合并 (DSU on Tree)
- 图论 - 第 19 页：最近公共祖先 (LCA)
- 图论 - 第 20 页：树直径
- 图论 - 第 22 页：树重心
- 图论 - 第 23 页：点分治 (Centroid Decomposition)
- 图论 - 第 24 页：虚树
- 图论 - 第 25 页：重链剖分 (Heavy-Light Decomposition)
- 图论 - 第 27 页：欧拉路径与欧拉回路
- 图论 - 第 28 页：Dinic 算法
- 图论 - 第 30 页：Edmonds-Karp (EK) 算法
- 图论 - 第 31 页：Hall 定理 (Hall's Marriage Theorem)
- 图论 - 第 32 页：费用流 (MCMF)
- 图论 - 第 35 页：圆方树 (Block-Cut Tree)
- 图论 - 第 35 页：强连通分量
- 图论 - 第 36 页：点双连通分量 (V-BCC)
- 图论 - 第 39 页：边双连通分量 (e-BCC)
- 字符串 - 第 41 页：AC 自动机
- 字符串 - 第 43 页：KMP 算法
- 字符串 - 第 44 页：Manacher
- 字符串 - 第 45 页：Trie 树
- 字符串 - 第 49 页：后缀数组（SA）
- 字符串 - 第 49 页：后缀自动机（SAM）
- 字符串 - 第 50 页：字符串哈希 (String Hash)
- 字符串 - 第 51 页：扩展 KMP（Z 函数）
- 数据结构 - 第 52 页：ST 表
- 数据结构 - 第 52 页：主席树
- 数据结构 - 第 54 页：可撤销并查集
- 数据结构 - 第 55 页：并查集
- 数据结构 - 第 56 页：权值线段树
- 数据结构 - 第 59 页：笛卡尔树
- 数据结构 - 第 60 页：线段树
- 数据结构 - 第 64 页：线段树分治
- 数据结构 - 第 66 页：线段树合并与分裂 (Segment Tree Merge & Split)
- 数据结构 - 第 68 页：线段树套线段树
- 数据结构 - 第 71 页：线段树的特殊应用
- 数据结构 - 第 73 页：莫队
- 数论 - 第 74 页：int128类定义
- 数论 - 第 75 页：中国剩余定理（CRT）
- 数论 - 第 76 页：二项式反演
- 数论 - 第 77 页：快速傅里叶变换 (FFT)
- 数论 - 第 78 页：快速数论变换
- 数论 - 第 79 页：扩展欧几里得算法（exgcd）
- 数论 - 第 80 页：拉格朗日插值
- 数论 - 第 81 页：指数生成函数 (EGF) 与有标号计数
- 数论 - 第 82 页：数论分块（整除分块）
- 数论 - 第 82 页：普通生成函数 (OGF) 与序列构造
- 数论 - 第 84 页：狄利克雷卷积
- 数论 - 第 84 页：狄利克雷生成函数 (DGF) 与算术函数
- 数论 - 第 85 页：矩阵快速幂
- 数论 - 第 86 页：线性基
- 数论 - 第 88 页：线性筛（欧拉筛）
- 数论 - 第 89 页：莫比乌斯反演
- 杂项 - 第 90 页：字符串哈希 (String Hash)

- 杂项 - 第 91 页：异或哈希 (Xor Hashing)
- 杂项 - 第 91 页：扫描线 (Scanning Line)
- 杂项 - 第 93 页：指数哈希
- 杂项 - 第 93 页：随机打乱数组
- 其他 - 第 93 页：计算几何

Ubuntu ICPCOS

创建run.sh

```
#!/bin/bash 或者 #!/bin/sh
target=$1
src="${target}.cpp"
g++ -std=gnu++20 -O2 -Wall -Wextra "$src" -o "$target" && \
./"$target" 2>&1 | tee "${target}.txt"
```

随后chmod +x ./run.sh

./run.sh A

位运算

```
popcount()//1的个数
countl_zero()//前导零
countr_zero()//后缀零
countl_one()//前导1
countr_one()//后缀1
has_single_bit()//是否只有一个1
bit_floor()        //向下取整到最近的2的幂
bit_ceil()         //向上取整到最近的2的幂

__builtin_popcount();//1的个数
__builtin_popcountll();
__builtin_clz();//前导零
__builtin_clzll();//ull
__builtin_ctz()//后缀零
__builtin_ctzll();//ull
```

向上向下取整

```
int Ceil(int n1,int n2) {
    if (n2 == 0) return 0;
    int x=n1/n2;
    int r=n1%n2;
    if (r != 0 and !((n1 < 0)^(n2 < 0))) x++;
    return x;
}

int Floor(int n1,int n2) {
    if (n2 == 0) return 0;
    int x=n1/n2;
    int r=n1%n2;
    if (r != 0 and ((n1 < 0)^(n2 < 0))) x--;
    return x;
}
```

随机数

```
int randINT(int l, int r) {
    static mt19937 gen(chrono::steady_clock::now().time_since_epoch().count());
    uniform_int_distribution<> distrib(l, r);
    return distrib(gen);
}
```

三分查找

```
        //查找区间最大值,要求f是严格单峰
        auto f=[&](int x) {

        };

        int lo,hi,res;
    while (hi-lo > 3) {
        int m1=lo+(hi-lo)/3;
        int m2=hi-(hi-lo)/3;
        if (f(m1) < f(m2)) lo=m1+1;
        else hi=m2-1;
    }
    res=f(lo);
    for (int i=lo+1;i<=hi;i++){
        res=max(res,f(i));
    }
```

SOS DP

定义

SOS DP（Sum Over Subsets DP）用于同时求出每个集合的所有子集或超集的权值和，本质是布尔格上的高维前缀和。设状态由 `n` 位二进制掩码表示，可将枚举每个状态全部子集的 $O(3^n)$ 优化为 $O(n2^n)$。

实现

子集求和维护

$$g[S]=\sum_{T\subseteq S} f[T]$$

即求每个集合的所有子集权值和。

初始令 `g = f`，依次枚举每一位 `bit`。当集合 `mask` 含有该位时，将未选择该位的状态 `mask ^ (1 << bit)` 合并到 `mask`。超集求和方向相反：当 `mask` 不含当前位时，将包含该位的状态合并到 `mask`。

掩码从 `0` 开始，输入数组长度必须恰好为 2^n。模板中的类型 `T` 需要支持加法；若答案可能超过类型范围，应改用模数类、`long long` 或 `__int128`。

代码

```
struct SOSDP
{
    vector<int> sum(vector<int> &a, int n)
    {
        int size = 1 << n;
        for (int bit = 0; bit < n; bit++)
        {
            for (int mask = 0; mask < size; mask++)
            {
                if (mask >> bit & 1)
                    a[mask] += f[mask ^ (1LL << bit)];
            }
        }
    }
};
```

时间复杂度为 $O(n2^n)$，额外空间复杂度为 $O(1)$；返回结果所占空间为 $O(2^n)$。

01背包问题

定义

给你n个物品，其中第i个物品的重量和价值为w_i,v_i,已经背包的总容量W。
求背包最多可以容纳的总价值。
时间复杂度为$O(N^2)$

实现

我们设DP状态为$dp[i][j]$在只能在前i个的物品中选择的情况下，容量为j的背包的最大价值。我们可以得到一些状态转移方程：

$$dp[i][j]=max(dp[i-1][j],dp[i-1][j-w_i]+v_i)$$

用滚动数组优化后的方程为：

$$dp[j] = max(dp[j], dp[j - w_i] + v_i)$$

代码

```cpp
#include <bits/stdc++.h>
using namespace std;

signed main()
{
    int n, v;
    cin >> n >> v;
    vector<array<int, 2>> a(n + 1);
    for (int i = 1; i <= n; i++)
        cin >> a[i][0] >> a[i][1];
    vector<int> dp(v + 1);
    for (int i = 1; i <= n; i++)
    {
        auto [cost, val] = a[i];
        for (int j = v; j >= cost; j--)
            dp[j] = max(dp[j], dp[j - cost] + val);
    }
    cout << dp[v] << '\n';
    return 0;
}
```

多重背包问题

定义

与01背包问题和完全背包问题不同的是，每一种物品的数量都只有k_i的。

实现

一个很朴素的想法就是：把「每种物品选 k_i 次」等价转换为「有 k_i 个相同的物品，每个物品选一次」。这样就转换成了一个 0-1 背包模型，套用上文所述的方法就可已解决。状态转移方程如下：

$$dp[i][j] = \max_{k=0}^{k_i}(dp[i][j - k \times w_i] + v_i \times k)$$

代码

```cpp
#include <bits/stdc++.h>
using namespace std;

signed main()
{
    int n, m;
    cin >> n >> m;
    vector<int> v(n + 1), w(n + 1), s(n + 1);
    vector<vector<int>> dp(n + 1, vector<int>(m + 1));
    for (int i = 1; i <= n; i++)
        cin >>
            v[i] >> w[i] >> s[i];
    for (int i = 1; i <= n; i++)
        for (int j = 0; j <= m; j++)
            for (int k = 0; k * v[i] <= j && k <= s[i]; k++)
                dp[i][j] = max(dp[i][j], dp[i - 1][j - k * v[i]] + k * w[i]);
    cout << dp[n][m] << endl;
    return 0;
}
```

多重背包问题的二进制优化

实现

考虑优化。我们仍考虑把多重背包转化成 0-1 背包模型来求解。
我们讲一钟物品分别表示由 2^j 个单个物品「捆绑」而成的大物品。

代码

```
#include <bits/stdc++.h>
#define int long long
using namespace std;

signed main()
{
    int n, W;
    cin >> n >> W;
    vector<array<int, 3>> its(n + 1);
    for (int i = 1; i <= n; i++)
    {
        cin >> its[i][0] >> its[i][1] >> its[i][2];
    }
    vector<int> dp(W + 1);
    for (int i = 1; i <= n; i++)
    {
        auto [val, cost, cnt] = its[i];
        for (int k = 1; k <= cnt; k *= 2)
        {
            for (int i = W; i >= k * cost; i--)
                dp[i] = max(dp[i], dp[i - k * cost] + val * k);
            cnt -= k;
        }
        int k = cnt;
        for (int i = W; i >= k * cost; i--)
            dp[i] = max(dp[i - k * cost] + val * k, dp[i]);
    }
    cout << dp[W] << '\n';
    return 0;
}
```

完全背包问题

定义

与01背包问题不同的是，每一种物品的数量都是无限的。

实现

我们设DP状态为$dp[i][j]$在只能在前i个的物品中选择的情况下，容量为j的背包的最大价值。
我们考虑一个朴素的做法：

$$dp[i][j] = \max_{k=0}^{+\infty}(dp[i][j - k \times w_i] + v_i \times k)$$

我们考虑一个简单的优化，对于$dp[i][j]$可以通过$dp[i][j - w_i]$来转移，因此我们可以得到：

$$dp[i][j] = max(dp[i][j - 1], dp[i][j - w_i] + v_i)$$

同样我们也可以用滚动数组去优化空间复杂度

代码

```
#include <bits/stdc++.h>
using namespace std;

signed main()
{
    int n, v;
    cin >> n >> v;
    vector<array<int, 2>> a(n + 1);
    for (int i = 1; i <= n; i++)
        cin >> a[i][0] >> a[i][1];
    vector<int> dp(v + 1);
    for (int i = 1; i <= n; i++)
    {
        auto [cost, val] = a[i];
        for (int j = cost; j <= v; j++)
            dp[j] = max(dp[j], dp[j - cost] + val);
    }
    cout << dp[v] << '\n';
    return 0;
}
```

Kruskal 重构树

定义

在 Kruskal 算法合并 u, v 所在的连通块时，不直接连边，而是新建一个节点 $node$，令 $node$ 的权值为该边的边权，并让 u, v 所在树的根节点成为 $node$ 的左右儿子。最终形成一棵拥有 $2n - 1$ 个节点的二叉树（原图连通情况下）。

性质：

1. 是一棵大根堆（如果是最小生成树重构）。
2. 原图中 u, v 路径上最大边权的最小值，等于重构树中 $LCA(u, v)$ 的点权。
3. 叶子节点均为原图节点，非叶子节点均为原图的边。

实现

采用 `struct` 封装，包含并查集初始化与重构树构建过程。

代码

```
struct DSU
{
    vector<int> dsu;
    DSU(int n)
    {
        dsu.assign(n, 0);
        iota(dsu.begin(), dsu.end(), 0);
    }
    int find(int x)
    {
        return dsu[x] == x ? x : dsu[x] = find(dsu[x]);
    }
};
struct KruTree
{
    int n, cur;
    vector<array<int, 3>> e;
    vector<vector<int>> g;
    vector<int> val;
    vector<vector<int>> fa;
    vector<int> dep;
    DSU dsu;
    KruTree(int n) : n(n), cur(n), e(), g(2 * n + 1, vector<int>{}), val(2 * n + 1), fa(2 * n + 1, vector<int>(31)), dep(2 * n + 1), dsu(2 * n + 1) { dep[0] = -1
    void add(int u, int v, int w)
    {
        e.pb({w, u, v});
    }
    void work()
    {
        sort(e.begin(), e.end());
        for (auto [w, u, v] : e)
        {
            int fu = dsu.find(u), fv = dsu.find(v);
            if (fu == fv)
                continue;
            cur++;
            dsu.dsu[fu] = dsu.dsu[fv] = cur;
            val[cur] = w;
            g[cur].pb(fu);
            g[cur].pb(fv);
        }
        for (int i = 1; i <= cur; i++)
        {
            if (dsu.find(i) == i)
                DFS(i, 0);
        }
    }
    void DFS(int u, int f)
    {
        dep[u] = dep[f] + 1;
        fa[u][0] = f;
        for (int i = 1; i <= 30; i++)
        {
            fa[u][i] = fa[fa[u][i - 1]][i - 1];
        }
        for (auto v : g[u])
        {
            if (v == f)
                continue;
            DFS(v, u);
        }
    }
    int lca(int u, int v)
    {
        if (dsu.find(u) != dsu.find(v))
            return -1;
        if (dep[u] > dep[v])
            swap(u, v);
        int d = dep[v] - dep[u];
        for (int i = 0; i <= 30; i++)
        {
            if (d >> i & 1)
                v = fa[v][i];
        }
        if (u == v)
            return val[u];
        for (int i = 30; i >= 0; i--)
        {
            if (fa[u][i] != fa[v][i])
            {
                u = fa[u][i];
                v = fa[v][i];
            }
        }
        return val[fa[u][0]];
    }
};
```

最小生成树

定义

最小生成树 (Minimum Spanning Tree, MST)：
在一个连通的加权无向图中，找到一个边的子集，使得这些边构成的树包含图中的所有顶点，并且这些边的权值之和尽可能小。如果图不连通，则不存在最小生成树（此时可求最小生成森林）。

实现

求最小生成树主要有两种经典算法：

- **Prim 算法 (普里姆算法)**：基于顶点贪心。每次选择距离当前生成树集合最近且未被访问的节点加入集合。配合优先队列（堆优化）的时间复杂度为 $O(M \log N)$，更适合**稠密图**。
- **Kruskal 算法 (克鲁斯卡尔算法)**：基于边贪心。将所有边按权值从小到大排序后遍历，利用**并查集**判断加入该边是否会形成环。时间复杂度为 $O(M \log M)$，更适合**稀疏图**。

代码

方式一：Prim 算法 (优先队列优化)

```cpp
#include <bits/stdc++.h>
#define pb push_back
using namespace std;

const int inf = 1e9;
void solve()
{
    int n, m;
    cin >> n >> m;
    vector<vector<array<int, 2>>> g(n + 1);
    for (int i = 1; i <= m; i++)
    {
        int u, v, w;
        cin >> u >> v >> w;
        g[u].pb({v, w});
        g[v].pb({u, w});
    }
    vector<int> vis(n + 1), dis(n + 1, inf);
    int ans = 0, cnt = 0;
    dis[1] = 0;
    priority_queue<array<int, 2>, vector<array<int, 2>>, greater<array<int, 2>>> pq;
    pq.push({0, 1});
    while (pq.size())
    {
        auto [d, u] = pq.top();
        pq.pop();
        if (vis[u])
            continue;
        vis[u] = 1;
        cnt++;
        ans += d;
        for (auto [v, w] : g[u])
        {
            if (dis[v] > w)
            {
                dis[v] = w;
                pq.push({w, v});
            }
        }
    }
    if (cnt != n)
        cout << "orz\n";
    else
        cout << ans << '\n';
}
signed main()
{
    ios::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

方式二：Kruskal 算法

```
#include <bits/stdc++.h>
#define pb push_back
using namespace std;

struct DSU {
          vector<int> f,siz;

          DSU() {}
          DSU(int n) {init(n);}

          void init(int n) {
                    f.resize(n);
                    iota(f.begin(),f.end(),0);
                    siz.assign(n,1);
          }

          int find(int x) {
                    while (x != f[x]) {
                              x = f[x] = f[f[x]];
                    }
                    return x;
          }

          bool merge(int x,int y) {
                    x=find(x);
                    y=find(y);
                    if (x == y) return 0;
                    siz[x] += siz[y];
                    f[y]=x;
                    return 1;
          }

          int esize(int x) {
                    return siz[find(x)];
          }

          int count() {
                    unordered_set<int> roots;
                    for(int i=0;i<f.size();i++) roots.insert(find(i));
                    return roots.size();
          }
};
void solve()
{
    int n, m;
    cin >> n >> m;
    vector<array<int,3>>e(m);
    for(auto &[w,u,v]:e)
        cin>>u>>v>>w;
    ranges::sort(e);
    DSU dsu(n+1);
    int ans = 0, cnt = 0;
    for(auto [w,u,v]:e){
        if(dsu.merge(u,v)){
            ans+=w,cnt++;
        }
    }
    if (cnt != n - 1)
        cout << "orz\n";
    else
        cout << ans << '\n';
}

signed main()
{
    ios::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

Bellman–Ford 算法

功能

- Bellman–Ford 算法是一种基于松弛（relax）操作的最短路算法，可以求出有负权的图的最短路。（Dijkstra 算法也会用到松弛操作）
- Bellman–Ford 算法也可以对最短路不存在的情况进行判断。
- Bellman–Ford 算法的时间复杂度是$O(nm)$。（它还有一种使用对列优化的算法SPFA算法）

实现

先介绍算法中的松弛操作：
对于边(u,v),松弛操作对应着一下式子：

$$dis(v) = min(dis(v), dis(u) + w(u, v))$$

Bellman–Ford 算法所做的，就是不断尝试对图上每一条边进行松弛。我们每进行一轮循环，就对图上所有的边都尝试进行一次松弛操作，当一次循环中没有成功的松弛操作时，算法停止。
每次循环的复杂度为$O(m)$，因为一次操作最多会使最短路变数$+1$,所以最多进行$n-1$次，那么总的时间复杂度就是$O(nm)$。

但还有一种情况，如果从S 点出发，抵达一个负环时，松弛操作会无休止地进行下去。注意到前面的论证中已经说明了，对于最短路存在的图，松弛操作最多只会执行 $n-1$ 轮，因此如果第 n 轮循环时仍然存在能松弛的边，说明从
S 点出发，能够抵达一个负环。

代码

```cpp
#include <bits/stdc++.h>
using namespace std;

const int N = 505, M = 1e4 + 10, INF = 0x3f3f3f3f;
struct edge
{
    int a, b, v;
} e[M];
int n, m, k;
int d[N], back_d[N];
int bellman_ford()
{
    memset(d, 0x3f, sizeof(d));
    d[1] = 0;
    for (int i = 1; i <= k; i++)
    {
        memcpy(back_d, d, sizeof(d));
        for (int j = 1; j <= m; j++)
        {
            int a = e[j].a, b = e[j].b, v = e[j].v;
            d[b] = min(d[b], back_d[a] + v);
        }
    }
    return d[n];
}
signed main()
{
    cin >> n >> m >> k;
    for (int i = 1; i <= m; i++)
    {
        cin >> e[i].a >> e[i].b >> e[i].v;
    }
    int ans = bellman_ford();
    if (ans > INF / 2)
        cout << "impossible" << endl;
    else
        cout << ans << endl;
    return 0;
}
```

Dijkstra 算法

功能

- Dijkstra 算法是一种求解 **非负权图** 上单源最短路径的算法。
- 朴素Dijkstra 算法的时间复杂度为$O(N^2)$,堆优化Dijkstra 算法的时间复杂度为$O(NlogN)$

实现

将结点分成两个集合：已确定最短路长度的点集（记为 S 集合）的和未确定最短路长度的点集（记为 T 集合）。一开始所有的点都属于 T 集合。

初始化$dis[s]=0$,其他点的 dis 均为 $+\infty$

然后重复一下操作：

1. 从T 集合中选取一个最短路长度最小的点，移到集合S 中。
2. 在对刚刚被加入S 集合的结点的所有出边执行松弛操作。
   直到 T 集合为空，算法结束。

在稀疏图中，$m=O(n)$,堆优化般的Dij具有较大优势；而在稠密图中 $m=O(n^2)$则使用朴素方法更优

代码

1. 朴素Dijkstra 算法

```cpp
#include <bits/stdc++.h>
using namespace std;

const int N = 505, M = 1e5 + 10, INF = 0x3f3f3f3f;
int n, m;
int g[N][N];
int d[N];
bool st[N];
int dijkstra()
{
    memset(d, 0x3f, sizeof(d));
    d[1] = 0;
    for (int i = 1; i <= n;i++){
        int t = -1;
        for (int j = 1; j <= n;j++)
            if(!st[j]&&(t==-1||d[j]<d[t]))
                t = j;
        st[t] = 1;
        for (int i = 1; i <= n;i++){
            d[i] = min(d[t] + g[t][i], d[i]);
        }
    }
    if(d[n]==INF)
        return -1;
    else
        return d[n];
}
signed main()
{
    cin >> n >> m;
    memset(g, 0x3f, sizeof(g));
    for (int i = 1; i <= m; i++)
    {
        int a, b, z;
        cin >> a >> b >> z;
        g[a][b] = min(z, g[a][b]);
    }
    cout << dijkstra() << endl;
    return 0;
}
```

2. 堆优化Dijkstra 算法

```cpp
#include <bits/stdc++.h>
using namespace std;

const int maxn = 1e6 + 10, INF = 0x3f3f3f3f;
typedef pair<int, int> PII;
int n, m;
vector<PII> g[maxn];
int d[maxn];
bool st[maxn];
int dijkstra()
{
    priority_queue<PII, vector<PII>, greater<PII>> h;
    memset(d, 0x3f, sizeof(d));
    d[1] = 0;
    h.push({0,1});
    while (h.size())
    {
        auto [dpos, pos] = h.top();
        h.pop();
        if (st[pos])
            continue;
        st[pos] = 1;
        for (auto [son, dson] : g[pos])
        {
            if (d[son] > dpos + dson)
            {
                d[son] = dpos + dson;
                h.push({d[son], son});
            }
        }
    }
    if (d[n] == INF)
        return -1;
    else
        return d[n];
}
signed main()
{
    cin >> n >> m;
    for (int i = 1; i <= m; i++)
    {
        int a, b, c;
        cin >> a >> b >> c;
        g[a].push_back({b, c});
    }
    cout << dijkstra() << endl;
    return 0;
}
```

Floyd 算法

功能

- 是用来求任意两个结点之间的最短路的。
- 复杂度比较高（时间复杂度是 $O(N^3)$ ，空间复杂度是 $O(N^2)$），但是常数小，容易实现（只有三个 `for` ）。
- 适用于任何图，不管有向无向，边权正负，但是最短路必须存在。（*不能有个负环*）

实现

定义一个数组$f[k][x][y]$,表示只用节点1 到节点 k 从 x 走到 y 的最短路。
显然$f[n][x][y]$ 为所求的最短路，且我们可以得到它的状态转移为：

$$f[k][x][y] = min(f[k-1][x][y], f[k-1][x][k] + f[k-1][k][y])$$

此时我们可以得到当前做法的空间复杂度为$O(N^3)$,因为第一维对结果并没有影响，所以可以省略，于是可以改成：

$$f[x][y] = min(f[x][y], f[k][y])$$

代码

```
#include <bits/stdc++.h>
using namespace std;

const int N = 205, INF = 0x3f3f3f3f;
int n, m, k;
int d[N][N];
void Floyd()
{
    for (int k = 1; k <= n; k++)
        for (int i = 1; i <= n; i++)
            for (int j = 1; j <= n; j++)
                d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
}
signed main()
{
    cin >> n >> m >> k;
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= n; j++)
            if (i == j)
                d[i][j] = 0;
            else
                d[i][j] = INF;
    while (m--)
    {
        int a, b, c;
        cin >> a >> b >> c;
        d[a][b] = min(d[a][b], c);
    }
    Floyd();
    while (k--)
    {
        int a, b;
        cin >> a >> b;
        if (d[a][b] > INF / 2)
            cout << "impossible" << endl;
        else
            cout << d[a][b] << endl;
    }
    return 0;
}
```

Johnson 全源最短路径算法

定义

Johnson 算法用于求解带负权边（但无负环）的有向图全源最短路径。它通过给每个节点赋予一个势能值 $h(u)$，将原边权 $w(u,v)$ 重塑为非负边权 $\hat{w}(u,v) = w(u,v) + h(u) - h(v)$，从而允许对每个源点调用 Dijkstra 算法。

实现 (Structure-based)

代码

```
struct johnson
{
    const int inf = 1e9;
    struct _edge
    {
        int to;
        int val;
        _edge(int to, int val) : to(to), val(val) {};
    };
    int n;
    vector<_edge> e;
    vector<vector<int>> g, D;
    vector<int> h;
    johnson(int n) : n(n)
    {
        init(n + 1);
    }
    void init(int n)
    {
        e.clear();
        g.assign(n, {});
        D.assign(n, vector<int>(n, inf));
    }
    void add(int u, int v, int w)
    {
        g[u].pb(e.size());
        e.pb({v, w});
    }
    int spfa()
    {
        for (int i = 1; i <= n; i++)
        {
            add(0, i, 0);
        }
        queue<int> q;
        h.assign(n + 1, inf);
        vector<int> vis(n + 1);
        vector<int> cnt(n + 1);
        h[0] = 0;
        vis[0] = 1;
        q.push(0);
        while (q.size())
        {
            auto u = q.front();
            q.pop();
            vis[u] = 0;
            for (int idx : g[u])
            {
                auto [v, w] = e[idx];
                if (h[v] > h[u] + w)
                {
                    h[v] = h[u] + w;
                    cnt[v] = cnt[u] + 1;
                    if (cnt[v] > n)
                        return 0;
                    if (!vis[v])
                        q.push(v), vis[v] = 1;
                }
            }
        }
        return 1;
    }
    void dij(int s)
    {
        priority_queue<array<int, 2>, vector<array<int, 2>>, greater<array<int, 2>>> pq;
        vector<int> dis(n + 1, inf);
        dis[s] = 0;
        pq.push({0, s});
        while (pq.size())
        {
            auto [d, u] = pq.top();
            pq.pop();
            if (dis[u] != d)
                continue;
            for (auto idx : g[u])
            {
                auto [v, w] = e[idx];
                if (dis[v] > d + h[u] - h[v] + w)
                {
                    dis[v] = d + h[u] - h[v] + w;
                    pq.push({dis[v], v});
                }
            }
        }
        for (int i = 1; i <= n; i++)
        {
            if (dis[i] > inf / 2)
                D[s][i] = inf;
```

```
            else
                D[s][i] = dis[i] - h[s] + h[i];
        }
    }
    bool work()
    {
        if (!spfa())
            return 0;
        for (int i = 1; i <= n; i++)
        {
            dij(i);
        }
        return 1;
    }
    int query(int s, int t)
    {
        return D[s][t];
    }
};
```

SPFA算法

功能

- 同样的SPFA算法也可以判断负环，只需记录最短路经过了多少条边，当经过了至少 n 条边时,说明存在一个负环。
- 大多时候SPFA算法跑得很快，但其最坏情况的时间复杂度可以到达 $O(nm)$ ，在没有负权边的时候最好还是使用 Dijkstra 算法 。

实现

Bellman–Ford 算法中很多时候我们并不需要很多松弛操作，很显然，只有上一次被松弛的结点，所连接的边，才有可能引起下一次的松弛操作。因此我们可以用队列维护那些点可能*会引起松弛操作* 就可以只访问必要的边了。

代码

```
#include <bits/stdc++.h>
using namespace std;

const int maxn = 1e6 + 10, INF = 0x3f3f3f3f;
typedef pair<int, int> PII;
int n, m;
vector<PII> g[maxn];
int d[maxn];
bool st[maxn];
int spfa()
{
    memset(d, 0x3f, sizeof(d));
    d[1] = 0;
    st[1] = 1;
    queue<int> q;
    q.push(1);
    while (q.size())
    {
        int t = q.front();
        q.pop();
        st[t] = 0;
        for (auto [son, dson] : g[t])
        {
            if (d[son] > d[t] + dson)
            {
                d[son] = d[t] + dson;
                if (!st[son])
                {
                    q.push(son);
                }
            }
        }
    }
    return d[n];
}
signed main()
{
    cin >> n >> m;
    for (int i = 1; i <= m; i++)
    {
        int a, b, c;
        cin >> a >> b >> c;
        g[a].push_back({b, c});
    }
    int t = spfa();
    if (t >INF/2)
        cout << "impossible" << endl;
    else
        cout << t << endl;
    return 0;
}
```

差分约束

定义

差分约束系统是一种特殊的 n 元一次不等式组，它包含 n 个变量 $x_1,x_2,x_3,\ldots,x_n$ 以及m个约束条件，每个约束条件是由两个变量做差构成的，$eg:x_i-x_j\le c_k$ *（并且 c_k 是常数）*，我们需要找到一组解使得约束条件成立，否则判断出无解。

在差分约束系统中每个约束条件 $x_i-x_j\le c_k$ 都可以变形为 $x_i\le c_k+x_j$ ，这与单源最短路中的三角不等式 $dis[y]\le dis[x]+z$ 非常相似。因此，我们可以把变量 x_i 看作图中的一个节点，而对于每一个约束条件 $x_i-x_j\le c_k$ 看作是一条从 x_j 向x_i 连一条长度为 c_k 的有向边。

实现

设$dis[0]=0$ 并向每个点连一条长度为0的边（添加一个源点0，保证每个点都会被遍历到），跑单元最短路，如果图中存在负环，则该差分约束系统无解，否则，$x_i=dis[i]$ 为差分系统的一组解。

代码

```
#include <bits/stdc++.h>
#define pb push_back
#define int long long
using namespace std;


// https://www.luogu.com.cn/problem/P1993
const int inf = 1e18;
void solve()
{
    int n, m;
    cin >> n >> m;
    vector<vector<array<int, 2>>> g(n + 1);
    for (int i = 1; i <= m; i++)
    {
        int op;
        cin >> op;
        if (op == 1)
        {
            int a, b, c;
            cin >> a >> b >> c;
            g[a].pb({b, -c});
        }
        else if (op == 2)
        {
            int a, b, c;
            cin >> a >> b >> c;
            g[b].pb({a, c});
        }
        else
        {
            int a, b;
            cin >> a >> b;
            g[a].pb({b, 0});
            g[b].pb({a, 0});
        }
    }
    for (int i = 1; i <= n; i++)
        g[0].pb({i, 0});
    vector<int> dis(n + 1, inf);
    auto spfa = [&]() -> bool
    {
        vector<int> vis(n + 1), cnt(n + 1);
        queue<int> q;
        dis[0] = 0;
        q.push(0);
        vis[0] = 1;
        while (q.size())
        {
            auto u = q.front();
            q.pop();
            vis[u] = 0;
            for (auto [v, w] : g[u])
            {
                if (dis[v] > dis[u] + w)
                {
                    cnt[v] = cnt[u] + 1;
                    dis[v] = dis[u] + w;
                    if (cnt[v] >= n)
                        return 0;
                    if (!vis[v])
                        q.push(v), vis[v] = 1;
                }
            }
        }
        return 1;
    };
    bool ok = spfa();
    cout << (ok ? "Yes\n" : "No\n");
}
signed main()
{
    ios::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

树上启发式合并 (DSU on Tree)

定义

用于处理不带修改的树上子树询问。通过优先遍历轻儿子并清除其贡献，最后遍历重儿子并保留其贡献，利用重链剖分的性质将复杂度优化至 $O(n \log n)$。

实现

使用结构体封装，通过 `dfs1` 预处理重儿子 `hson`， `dfs2` 进行启发式合并。

代码

```
#include <bits/stdc++.h>
#define int long long
using namespace std;

const int inf = 1e18;
void solve()
{
    int n;
    cin >> n;
    vector<int> c(n + 1);
    for (int i = 1; i <= n; i++)
        cin >> c[i];
    vector adj(n + 1, vector<int>());
    int root = -1;
    for (int i = 1; i < n; i++)
    {
        int u, v;
        cin >> u >> v;
        if (root == -1)
            root = u;
        adj[u].push_back(v);
        adj[v].push_back(u);
    }
    vector<int> son(n + 1), sz(n + 1);
    auto get_son = [&](auto &&self, int u, int f) -> void
    {
        sz[u] = 1;
        for (auto v : adj[u])
        {
            if (v == f)
                continue;
            self(self, v, u);
            sz[u] += sz[v];
            if (sz[son[u]] < sz[v])
                son[u] = v;
        }
    };
    get_son(get_son, root, 0);
    int flag = 0, sum = 0, ma = 0;
    vector<int> cnt(n + 1), ans(n + 1);
    auto update = [&](int u, int val)
    {
        cnt[c[u]] += val;
        if (cnt[c[u]] > ma)
        {
            ma = cnt[c[u]];
            sum = c[u];
        }
        else if (cnt[c[u]] == ma)
            sum += c[u];
    };
    auto init = [&]()
    { sum = 0, ma = 0; };
    auto count = [&](auto &&self, int u, int f, int val) -> void
    {
        update(u, val);
        for (auto v : adj[u])
        {
            if (v == f || v == flag)
                continue;
            self(self, v, u, val);
        }
    };
    auto dfs = [&](auto &&self, int u, int f, bool keep) -> void
    {
        for (auto v : adj[u])
        {
            if (v == son[u] || v == f)
                continue;
            self(self, v, u, 0);
        }
        if (son[u])
        {
            self(self, son[u], u, 1);
            flag = son[u];
        }
        count(count, u, f, 1);
        flag = 0;
        ans[u] = sum;
        if (!keep)
        {
            count(count, u, f, -1);
            init();
        }
    };
    dfs(dfs, root, 0, 0);
    for (int i = 1; i <= n; i++)
        cout << ans[i] << " \n"[i == n];
}
signed main()
{
```

```
    ios::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

最近公共祖先 (LCA)

朴素实现 (Naive)

通过跳指针直到两点相遇。适用于单次查询或树高较小的情况。

代码

```
struct LCA {
    int n;
    vector<int> d;
    vector<int> p;
    vector<vector<int>> adj;
    LCA(int n) : n(n), d(n + 1), p(n + 1), adj(n + 1) {}
    void add(int u, int v) {
        adj[u].push_back(v);
        adj[v].push_back(u);
    }
    void dfs(int u, int fa) {
        d[u] = d[fa] + 1;
        p[u] = fa;
        for (int v : adj[u]) {
            if (v != fa) dfs(v, u);
        }
    }
    int query(int u, int v) {
        if (d[u] < d[v]) swap(u, v);
        while (d[u] > d[v]) u = p[u];
        while (u != v) u = p[u], v = p[v];
        return u;
    }
};
```

倍增实现 (Binary Lifting)

利用 2^k 跳跃优化查找过程，预处理 $O(n \log n)$，单次查询 $O(\log n)$。

代码

C++

```cpp
struct LCA {
    int n, l;
    vector<int> d;
    vector<vector<int>> f;
    vector<vector<int>> adj;

    LCA(int n) : n(n), l(__lg(n) + 1), d(n + 1), f(n + 1, vector<int>(l, 0)), adj(n + 1) {}

    void add(int u, int v) {
        adj[u].push_back(v);
        adj[v].push_back(u);
    }

    void work(int rt = 1) {
        dfs(rt, 0);
    }

    void dfs(int u, int fa) {
        d[u] = d[fa] + 1;
        f[u][0] = fa;
        for (int i = 1; i < l; i++) {
            f[u][i] = f[f[u][i - 1]][i - 1];
        }
        for (int v : adj[u]) {
            if (v != fa) dfs(v, u);
        }
    }

    int query(int u, int v) {
        if (d[u] < d[v]) swap(u, v);
        for (int i = l - 1; i >= 0; i--) {
            if (d[f[u][i]] >= d[v]) u = f[u][i];
        }
        if (u == v) return u;
        for (int i = l - 1; i >= 0; i--) {
            if (f[u][i] != f[v][i]) {
                u = f[u][i];
                v = f[v][i];
            }
        }
        return f[u][0];
    }

    int dist(int u, int v) {
        return d[u] + d[v] - 2 * d[query(u, v)];
    }
};
```

树直径

定义

我们将树上任意两节点之间最长的简单路径即为树的「直径」。
显然一棵树可以有多条直径。

性质

当树上所有边权为正时，则树的所有直径中点重合。

做法

我们可以用两次 DFS ,或者树形dp 的方法去求出树的直径。

1. 两次DFS
   引理：在一棵树上，从任意节点 y 开始进行一次 DFS，到达的距离其最远的节点 z 必为直径的一端。*(证明可以使用反证法)*
   首先从任意节点 y 开始进行第一次 DFS，到达距离其最远的节点，记为 z，然后再从 z 开始做第二次 DFS，到达距离 z 最远的节点，记为 z'，则 $\delta(z,z')$ 即为树的直径。
2. 树形dp
   树形 DP 可以在存在负权边的情况下求解出树的直径。
   方法1:
   当我们记录1为树的根时，所能延伸的最长路径长度 $d1$ 与次长路径（与最长路径无公共边）长度 d_1，那么直径就是对于每一个点，该点 d_1+d_2 能取到的值中的最大值。
   方法2:
   定义$dp[u]$为以u为根的子树中，从u出发的最长路径，那么我们可以的到转移方程：

$$dp[u]=max(dp[u],dp[v]+w(u,v))$$

   其中在更新$dp[u]$之前，要先计算$d=max(d,dp[u]+dp[v]+w(u,v))$ 。

代码

两次dfs代码：

```cpp
#include <bits/stdc++.h>
#define pb push_back
using namespace std;

signed main()
{
    int n;
    cin >> n;
    vector<vector<int>> g(n + 1);
    for (int i = 1; i < n; i++)
    {
        int u, v;
        cin >> v >> u;
        g[u].pb(v);
        g[v].pb(u);
    }
    vector<int> dis(n + 1);
    int l = 1;
    auto dfs = [&](auto &&self, int u, int fa) -> void
    {
        for (auto v : g[u])
        {
            if (v == fa)
                continue;
            dis[v] = dis[u] + 1;
            if (dis[v] > dis[l])
                l = v;
            self(self, v, u);
        }
    };
    dfs(dfs, 1, 0);
    dis.assign(n + 1, 0);
    dfs(dfs, l, 0);
    cout << dis[l] << '\n';
    return 0;
}
```

树形dp代码1：

```cpp
#include <bits/stdc++.h>
#define pb push_back
using namespace std;

signed main()
{
    int n;
    cin >> n;
    vector<vector<int>> g(n + 1);
    for (int i = 1; i < n; i++)
    {
        int u, v;
        cin >> v >> u;
        g[u].pb(v);
        g[v].pb(u);
    }
    vector<int> d1(n + 1), d2(n + 1);
    auto dfs=[&](auto &&self, int u, int f) -> int
    {
        d1[u] = 0, d2[u] = 0;
        int d = 0;
        for (auto v : g[u])
        {
            if (v == f)
                continue;
            d = max(d, self(self, v, u));
            int tmp = d1[v] + 1;
            if (tmp > d1[u])
                d2[u] = d1[u], d1[u] = tmp;
            else if (tmp > d2[u])
                d2[u] = tmp;
        }
        return max(d, d1[u] + d2[u]);
    };

    cout << dfs(dfs, 1, 0) << '\n';
    return 0;
}
```

树形dp代码2:

```cpp
#include <bits/stdc++.h>
#define pb push_back
using namespace std;

signed main()
{
    int n;
    cin >> n;
    vector<vector<int>> g(n + 1);
    for (int i = 1; i < n; i++)
    {
        int u, v;
        cin >> v >> u;
        g[u].pb(v);
        g[v].pb(u);
    }
    vector<int> dp(n + 1);
    auto dfs = [&](auto &&self, int u, int f) -> int
    {
        int d = 0;
        for (auto v : g[u])
        {
            if (v == f)
                continue;
            d = max(d, self(self, v, u));
            d = max(d, dp[u] + dp[v] + 1);
            dp[u] = max(dp[u], dp[v] + 1);
        }
        return d;
    };

    cout << dfs(dfs, 1, 0) << '\n';
    return 0;
}
```

树重心

定义

在一个拥有 n 个节点的树中，如果删除某个节点 u 后，剩下的所有连通分量中最大的子树大小最小，则称节点 u 为树的重心。

- 树的重心最多有两个。
- 删除重心后，最大子树的大小 $max_sz \leq \lfloor n/2 \rfloor$。
- 所有点到树重心距离总和最小。

实现（dfs）

使用一次深度优先搜索（DFS）即可求出。通过 `sz[u]` 维护子树大小，并计算删除当前节点后的最大连通块。

代码

```cpp
#include <bits/stdc++.h>
using namespace std;

void solve()
{
    int n;
    cin >> n;
    vector adj(n + 1, vector<int>());
    for (int i = 1; i < n; i++)
    {
        int u, v;
        cin >> u >> v;
        adj[u].push_back(v);
        adj[v].push_back(u);
    }
    vector<int> sz(n + 1), dp(n + 1);
    vector<int> ans;
    auto dfs = [&](auto &&self, int u, int f) -> void
    {
        sz[u] = 1;
        dp[u] = 0;
        for (auto v : adj[u])
        {
            if (v == f)
                continue;
            self(self, v, u);
            sz[u] += sz[v];
            dp[u] = max(dp[u], sz[v]);
        }
        dp[u] = max(dp[u], n - sz[u]);
        if (dp[u] <= n / 2)
        {
            ans.push_back(u);
        }
    };
    dfs(dfs, 1, 0);
    cout << ans[0] << '\n';
}
```

实现（换根dp）

通过两次 DFS 实现：

1. **第一次 DFS**：计算以 0 为根时，每个节点的子树大小 $sz[u]$，并记录每个节点在向下方向的最大子树。
2. **第二次 DFS**：进行换根，计算每个节点向上方向（经过父亲节点）能到达的连通块大小，即 $n - sz[u]$。

代码

```
#include <bits/stdc++.h>
#define int long long
using namespace std;

const int mod = 998244353, N = 2e5;
void solve()
{
    int n;
    cin >> n;
    vector adj(n + 1, vector<int>());
    for (int i = 1; i < n; i++)
    {
        int u, v;
        cin >> u >> v;
        adj[u].push_back(v);
        adj[v].push_back(u);
    }
    vector<int> sz(n + 1), dp(n + 1), dep(n + 1, -1);
    dp[0] = 1e9;
    vector<int> ans;
    auto dfs1 = [&](auto &&self, int u, int f) -> void
    {
        sz[u] = 1;
        dep[u] = dep[f] + 1;
        for (auto v : adj[u])
        {
            if (v == f)
                continue;
            self(self, v, u);
            sz[u] += sz[v];
        }
        dp[1] += dep[u];
    };
    dfs1(dfs1, 1, 0);
    auto dfs2 = [&](auto &&self, int u, int f) -> void
    {
        for (auto v : adj[u])
        {
            if (v == f)
                continue;
            dp[v] = dp[u] + n - 2 * sz[v];
            self(self, v, u);
        }
    };
    dfs2(dfs2, 1, 0);
    cout << ranges::min_element(dp) - dp.begin() << ' ' << *ranges::min_element(dp) << '\n';
}
```

点分治 (Centroid Decomposition)

实现

通过递归寻找树的重心，将原问题分解为经过重心的路径问题和子树内的递归问题。

代码

```
struct CentroidDecomp {
    int n;
    std::vector<std::vector<int>> adj;
    std::vector<int> sz, vis;

    CentroidDecomp(int n_) : n(n_), adj(n_ + 1), sz(n_ + 1), vis(n_ + 1) {}

    void add(int u, int v) {
        adj[u].push_back(v);
        adj[v].push_back(u);
    }

    void get_sz(int u, int p) {
        sz[u] = 1;
        for (int v : adj[u]) {
            if (v != p && !vis[v]) {
                get_sz(v, u);
                sz[u] += sz[v];
            }
        }
    }

    int get_ct(int u, int p, int tot) {
        for (int v : adj[u]) {
            if (v != p && !vis[v] && sz[v] > tot / 2) {
                return get_ct(v, u, tot);
            }
        }
        return u;
    }

    void calc(int u) {
        // 在此处处理经过重心 u 的贡献
    }

    void solve(int u) {
        get_sz(u, 0);
        int ct = get_ct(u, 0, sz[u]);
        vis[ct] = 1;
        calc(ct); // 处理当前连通块
        for (int v : adj[ct]) {
            if (!vis[v]) solve(v);
        }
    }
};
```

虚树

定义

虚树保留关键点及连接它们所必需的 LCA，将原树上的最小连通子树压缩为规模至多 $2k-1$ 的树，其中 k 为去重后的关键点数。适用于只需处理少量关键点之间祖先关系或路径信息的树上询问。

实现

先按 DFS 序排序并去重关键点，再加入相邻关键点的 LCA；重新排序去重后，使用单调栈维护当前节点的祖先链并连接虚树边。 `p[i]` 表示虚树编号 `i` 对应的原树节点， `pos[u]` 表示原树节点 `u` 的虚树编号， `fa` 和 `adj` 均使用虚树编号。

以下组件依赖外部 `LCA` 类型： `dfn[u]` 为原树节点的 DFS 序， `query(u, v)` 返回 LCA， `check(u, v)` 判断 `u` 是否为 `v` 的祖先。原树节点使用 1 下标；重复关键点会被删除，空关键点集合会得到 `root = -1` 的空虚树。

代码

```
struct VirtualTree
{
    LCA &g;
    //check(u,v):u是否为v的祖先，dfn[u]<=dfn[v]&&out[u]>=dfn[v];

    int root = -1;
    std::vector<int> p;
    std::vector<int> fa;
    std::vector<std::vector<int>> adj;
    std::unordered_map<int, int> pos;

    VirtualTree(LCA &g) : g(g) {}

    void build(std::vector<int> key)
    {
        auto cmp = [&](int u, int v)
        {
            return g.dfn[u] < g.dfn[v];
        };

        std::ranges::sort(key, cmp);
        key.erase(std::unique(key.begin(), key.end()), key.end());

        root = -1;
        p.clear();
        fa.clear();
        adj.clear();
        pos.clear();
        if (key.empty())
            return;

        p = std::move(key);
        int k = static_cast<int>(p.size());
        for (int i = 0; i + 1 < k; i++)
        {
            p.push_back(g.query(p[i], p[i + 1]));
        }

        std::ranges::sort(p, cmp);
        p.erase(std::unique(p.begin(), p.end()), p.end());

        int n = static_cast<int>(p.size());
        fa.assign(n, -1);
        adj.assign(n, {});
        pos.reserve(p.size() * 2 + 1);
        for (int i = 0; i < n; i++)
        {
            pos[p[i]] = i;
        }

        std::vector<int> stk;
        for (int i = 0; i < n; i++)
        {
            while (!stk.empty() && !g.check(p[stk.back()], p[i]))
            {
                stk.pop_back();
            }

            if (stk.empty())
            {
                root = i;
            }
            else
            {
                fa[i] = stk.back();
                adj[fa[i]].push_back(i);
            }
            stk.push_back(i);
        }
    }
};
```

构造一次虚树的时间复杂度为 $O(k\log k + k\log n)$，额外空间复杂度为 $O(k)$。若 `check` 使用 DFS 进入、离开时间实现 $O(1)$ 祖先判断，单调栈部分为 $O(k)$；虚树边代表原树上的整段祖先路径，路径长度可通过 `g.dist(p[fa[v]], p[v])` 取得。

递归实现的 LCA 预处理可能在链形树上爆栈；数据规模较大时应改用迭代 DFS。LCA 的 `dfn` 容器应分配为 `n + 1`，并使用 `LCA g(n)`，不要依靠额外传入 `n + 1` 掩盖越界问题。

重链剖分 (Heavy-Light Decomposition)

实现

通过两次 DFS 预处理出重儿子、深度、父节点、子树大小以及链顶节点。结合线段树可维护路径及子树信息。

代码

```
struct HLD
{
    int n, root;
    vector<int> top, son, siz, dep, dfn, rnk, fa;
    vector<vector<int>> g;
    int cur;
    HLD(int n, int root) : n(n), root(root)
    {
        init();
    }
    void init()
    {
        g.assign(n, {});
        top.assign(n, 0);
        fa.assign(n, 0);
        son.assign(n, -1);
        rnk.assign(n, 0);
        dep.assign(n, 0);
        dfn.assign(n, 0);
        siz.assign(n, 0);
        dep[root] = 0;
        cur = 0;
    }
    void add(int u, int v)
    {
        g[u].push_back(v);
        g[v].push_back(u);
    }
    void dfs1(int u, int f)
    {
        fa[u] = f, dep[u] = (f == -1 ? -1 : dep[f]) + 1, siz[u] = 1;
        for (auto v : g[u])
        {
            if (v == f)
                continue;
            dfs1(v, u);
            siz[u] += siz[v];
            if (son[u] == -1 || siz[v] > siz[son[u]])
                son[u] = v;
        }
    }
    void dfs2(int u, int ftop)
    {
        dfn[u] = cur, rnk[cur] = u, top[u] = ftop;
        cur++;
        if (son[u] != -1)
            dfs2(son[u], ftop);
        for (auto v : g[u])
        {
            if (v == son[u] || v == fa[u])
                continue;
            dfs2(v, v);
        }
    }
    void work()
    {
        dfs1(root, -1);
        dfs2(root, root);
    }
    int lca(int u, int v)
    {
        while (top[u] != top[v])
        {
            if (dep[top[u]] > dep[top[v]])
                u = fa[top[u]];
            else
                v = fa[top[v]];
        }
        return dep[u] > dep[v] ? v : u;
    }
    vector<array<int, 2>> path(int u, int v, bool edge = 0)
    {
        vector<array<int, 2>> res;
        while (top[u] != top[v])
        {
            if (dep[top[u]] < dep[top[v]])
                swap(u, v);
            res.push_back({dfn[top[u]], dfn[u]});
            u = fa[top[u]];
        }
        if (u == v && edge)
            return res;
        if (dep[u] > dep[v])
            swap(u, v);
        res.push_back({dfn[u] + (edge ? 1 : 0), dfn[v]});
        return res;
    }
    array<int, 2> subtree(int u, bool edge = 0) const
    {
        return {dfn[u] + (edge ? 1 : 0), dfn[u] + siz[u] - 1};
```

```
    }
};
```

欧拉路径与欧拉回路

定义

欧拉路径经过图中每条边恰好一次，欧拉回路还要求起点与终点相同。判断时忽略孤立点，并要求所有非孤立点在对应的无向图中连通。

存在条件

- **无向图**：奇度点数量为 0 或 2 时存在欧拉路径；为 0 时存在欧拉回路，为 2 时路径必须从一个奇度点走到另一个奇度点。
- **有向图**：所有点入度等于出度时存在欧拉回路；或恰有一个点满足 $out = in + 1$、一个点满足 $in = out + 1$，其余点入度等于出度时存在欧拉路径。

实现

使用 Hierholzer 算法不断沿未使用边前进，无边可走时将当前点加入答案，最后反转序列。时间复杂度为 $O(n + m)$，空间复杂度为 $O(n + m)$，其中 n 为点数，m 为边数。

模板点编号为 $0 \sim n - 1$，支持重边和自环。`run()` 返回长度为 $m + 1$ 的点序列；不存在欧拉路径时返回空数组，无边图返回 `{0}`。

代码

```cpp
#include <bits/stdc++.h>
using namespace std;

struct euler
{
    vector<vector<int>> adj;
    vector<int> ind, out;
    vector<int> cur;
    int n;
    euler(int n) : n(n), adj(n), ind(n), out(n) {}
    void add(int u, int v)
    {
        adj[u].push_back(v);
        out[u]++, ind[v]++;
    };

    vector<int> work()
    {
        vector<int> res;
        int s = -1, t = -1;
        for (int i = 0; i < n; i++)
        {
            if (out[i] == ind[i] + 1)
            {
                if (s != -1)
                    return res;
                s = i;
            }
            else if (ind[i] == out[i] + 1)
            {
                if (t != -1)
                    return res;
                t = i;
            }
            else if (out[i] != ind[i])
                return res;
        }
        if (s == -1)
        {
            for (int i = 0; i < n; i++)
            {
                if (out[i] != 0)
                {
                    s = i;
                    break;
                }
            }
        }
        for (int i = 0; i < n; i++)
        {
            ranges::sort(adj[i]);
        }
        if (s == -1)
            return res;
        cur.assign(n, 0);
        auto dfs = [&](auto &&self, int u) -> void
        {
            for (int &i = cur[u]; i < adj[u].size();)
            {
                auto v = adj[u][i++];
                self(self, v);
            }
            res.push_back(u);
        };
        dfs(dfs, s);
        ranges::reverse(res);
        return res;
    }
};
```

Dinic 算法

Dinic 算法用于求解网络流中的**最大流问题**。它通过在分层图（Level Graph）上多次寻找增广路来优化效率，引入了**当前弧优化**和**多路增广**。

时间复杂度：一般图为 $O(V^2E)$，在二分图中为 $O(E\sqrt{V})$。

实现

基于分层图优化的 Dinic 算法。

代码

```
struct Flow
{
    struct _edge
    {
        int to;
        int cap;
        _edge(int to, int cap) : to(to), cap(cap) {};
    };
    const int inf = 1e18;
    int n;
    vector<_edge> e;
    vector<vector<int>> g;
    vector<int> cur, h;
    Flow() {};
    Flow(int n) : n(n)
    {
        init(n);
    }
    void init(int n)
    {
        e.clear();
        g.assign(n, {});
        cur.assign(n, 0);
        h.assign(n, -1);
    }
    void add(int u, int v, int c)
    {
        g[u].push_back(e.size());
        e.push_back({v, c});
        g[v].push_back(e.size());
        e.push_back({u, 0});
    }
    bool bfs(int s, int t)
    {
        h.assign(n, -1);
        queue<int> q;
        h[s] = 0;
        q.push(s);
        while (q.size())
        {
            auto u = q.front();
            q.pop();
            for (auto p : g[u])
            {
                auto [v, c] = e[p];
                if (c > 0 && h[v] == -1)
                {
                    h[v] = h[u] + 1;
                    if (v == t)
                        return 1;
                    q.push(v);
                }
            }
        }
        return 0;
    }
    int dfs(int u, int t, int f)
    {
        if (u == t)
            return f;
        int r = f;
        for (int &i = cur[u]; i < g[u].size(); i++)
        {
            int j = g[u][i];
            auto [v, c] = e[j];
            if (c > 0 && h[v] == h[u] + 1)
            {
                int nf = dfs(v, t, min(r, c));
                e[j].cap -= nf;
                e[j ^ 1].cap += nf;
                r -= nf;
                if (r == 0)
                    return f;
            }
        }
        return f - r;
    }
    int flow(int s, int t)
    {
        int ans = 0;
        while (bfs(s, t))
        {
            cur.assign(n, 0);
            ans += dfs(s, t, inf);
        }
        return ans;
    }
    vector<bool> minCut()
    {
        vector<bool> c(n);
        for (int i = 0; i < n; i++)
```

```
        {
            c[i] = (h[i] != -1);
        }
        return c;
    }
    struct Edge
    {
        int from;
        int to;
        int cap;
        int flow;
    };
    vector<Edge> edges()
    {
        vector<Edge> a;
        for (int i = 0; i < e.size(); i += 2)
        {
            Edge x;
            x.from = e[i + 1].to;
            x.to = e[i].to;
            x.cap = e[i].cap + e[i + 1].cap;
            x.flow = e[i + 1].cap;
            a.push_back(x);
        }
        return a;
    }
};
```

Edmonds-Karp (EK) 算法

Edmonds-Karp 算法通过不断在残量网络中寻找增广路（使用 BFS）来增加流量，直到不存在增广路为止。其时间复杂度为 $O(VE^2)$。

实现

代码

```
const int inf = 1e18;
struct EK
{
    int n;
    struct _edge
    {
        int to;
        int cap;
        _edge(int to, int cap) : to(to), cap(cap) {};
    };
    vector<_edge> e;
    vector<vector<int>> g;
    vector<int> pre, pre_e;

    EK() {};
    EK(int n) : n(n) { init(n); };
    void init(int n)
    {
        e.clear();
        g.assign(n, {});
        pre.assign(n, 0);
        pre_e.assign(n, 0);
    }
    void add(int u, int v, int c)
    {
        g[u].push_back(e.size());
        e.push_back({v, c});
        g[v].push_back(e.size());
        e.push_back({u, 0});
    }
    bool bfs(int s, int t)
    {
        vector<int> vis(n);
        queue<int> q;
        q.push(s);
        pre[s] = -1;
        pre_e[s] = -1;
        while (q.size())
        {
            auto u = q.front();
            q.pop();
            for (auto p : g[u])
            {
                auto [v, c] = e[p];
                if (vis[v] || c <= 0)
                    continue;
                vis[v] = 1;
                pre[v] = u;
                pre_e[v] = p;
                q.push(v);
                if (v == t)
                    return 1;
            }
        }
        return 0;
    }
    int flow(int s, int t)
    {
        int res = 0;
        while (bfs(s, t))
        {
            int f = inf;
            for (int v = t; v != s; v = pre[v])
            {
                int p = pre_e[v];
                f = min(f, e[p].cap);
            }
            for (int v = t; v != s; v = pre[v])
            {
                int p = pre_e[v];
                e[p].cap -= f;
                e[p ^ 1].cap += f;
            }
            res += f;
        }
        return res;
    }
};
```

Hall 定理 (Hall's Marriage Theorem)

定义

Hall 定理是二分图包含**完备匹配**的充分必要条件。

设二分图 $G = (V_1 \cup V_2, E)$，其中 $|V_1| \leq |V_2|$。

对于 V_1 的任意子集 $S \subseteq V_1$，设 $N(S)$ 为 S 中所有顶点的邻居集合。

图 G 存在一个匹配覆盖 V_1 中所有顶点的充分必要条件是：

$$\forall S \subseteq V_1, |N(S)| \geq |S|$$

推广（Hall 定理的推论）
二分图的最大匹配数为：

$$|V_1| - \max_{S \subseteq V_1}(|S| - |N(S)|)$$

其中 $\max(|S| - |N(S)|)$ 通常被称为二分图的 **缺额 (Deficiency)**。

实现

复杂度

- **理论判定**：通常涉及指数级数量的子集考察，复杂度为 $O(2^{|V_1|})$。
- **实际建模**：在题目中常转化为**网络流**（$O(E\sqrt{V})$）或通过**数据结构**（如线段树）维护序列上的区间 Hall 定理条件，复杂度通常为 $O(n \log n)$ 或 $O(n)$。

费用流 (MCMF)

定义

在网络流图中，每条边除了容量 $c(u, v)$ 外，还有单位流量的费用 $w(u, v)$。费用流算法旨在寻找从源点 S 到汇点 T 的最大流量中，总费用最小的流。

实现 (SPFA 增广路)

采用 SPFA 算法寻找最短路（单位费用为权值），通过 `pre` 数组记录路径进行增广。适用于含负权边但不含负权圈的图。

复杂度

$O(F \cdot E \log V)$ 或 $O(F \cdot VE)$，其中 F 是最大流量，E 是边数，V 是点数。

代码

```
struct MinCostFlow
{
    const int inf = 1e18;
    struct _edge
    {
        int to;
        int cap;
        int cost;
        _edge(int to, int cap, int cost) : to(to), cap(cap), cost(cost) {};
    };
    int n;
    vector<_edge> e;
    vector<vector<int>> g;
    vector<int> dis, h;
    vector<int> pre;
    MinCostFlow(int n) : n(n)
    {
        init(n);
    }
    void init(int n)
    {
        e.clear();
        g.assign(n, {});
    }
    void add(int u, int v, int w, int c)
    {
        g[u].push_back(e.size());
        e.pb({v, w, c});
        g[v].push_back(e.size());
        e.pb({u, 0, -c});
    }
    bool dij(int s, int t)
    {
        dis.assign(n, inf);
        pre.assign(n, -1);
        priority_queue<array<int, 2>, vector<array<int, 2>>, greater<array<int, 2>>> pq;
        dis[s] = 0;
        pq.push({0, s});
        while (pq.size())
        {
            auto [d, u] = pq.top();
            pq.pop();
            if (dis[u] != d)
                continue;
            for (auto idx : g[u])
            {
                auto [v, cap, w] = e[idx];
                if (cap > 0 && dis[v] > d + h[u] - h[v] + w)
                {
                    dis[v] = d + h[u] - h[v] + w;
                    pre[v] = idx;
                    pq.push({dis[v], v});
                }
            }
        }
        return dis[t] != inf;
    }
    array<int, 2> flow(int s, int t)
    {
        int f = 0;
        int cost = 0;
        h.assign(n, 0);
        while (dij(s, t))
        {
            for (int i = 0; i < n; i++)
            {
                h[i] += dis[i];
            }
            int nf = inf;
            for (int i = t; i != s; i = e[pre[i] ^ 1].to)
                nf = min(nf, e[pre[i]].cap);
            for (int i = t; i != s; i = e[pre[i] ^ 1].to)
                e[pre[i]].cap -= nf, e[pre[i] ^ 1].cap += nf;
            f += nf;
            cost += nf * h[t];
        }
        return array<int, 2>({f, cost});
    }
};
```

```cpp
struct MinCostFlow
{
    const int inf = 1e18;
    struct _edge
    {
        int to;
        int cap;
        int cost;
        _edge(int to, int cap, int cost) : to(to), cap(cap), cost(cost) {};
    };
    int n;
    vector<_edge> e;
    vector<vector<int>> g;
    vector<int> dis, h;
    vector<int> pre;
    MinCostFlow(int n) : n(n)
    {
        init(n);
    }
    void init(int n)
    {
        e.clear();
        g.assign(n, {});
    }
    void add(int u, int v, int w, int c)
    {
        g[u].push_back(e.size());
        e.pb({v, w, c});
        g[v].push_back(e.size());
        e.pb({u, 0, -c});
    }
    bool dij(int s, int t)
    {
        dis.assign(n, inf);
        pre.assign(n, -1);
        priority_queue<array<int, 2>, vector<array<int, 2>>, greater<array<int, 2>>> pq;
        dis[s] = 0;
        pq.push({0, s});
        while (pq.size())
        {
            auto [d, u] = pq.top();
            pq.pop();
            if (dis[u] != d)
                continue;
            for (auto idx : g[u])
            {
                auto [v, cap, w] = e[idx];
                if (cap > 0 && dis[v] > d + h[u] - h[v] + w)
                {
                    dis[v] = d + h[u] - h[v] + w;
                    pre[v] = idx;
                    pq.push({dis[v], v});
                }
            }
        }
        return dis[t] != inf;
    }
    void spfa(int s)
    {
        h.assign(n, inf);
        vector<bool> vis(n, false);
        queue<int> q;
        h[s] = 0;
        q.push(s);
        vis[s] = true;
        while (!q.empty())
        {
            int u = q.front();
            q.pop();
            vis[u] = false;
            for (auto idx : g[u])
            {
                auto [v, cap, w] = e[idx];
                if (cap > 0 && h[v] > h[u] + w)
                {
                    h[v] = h[u] + w;
                    if (!vis[v])
                    {
                        vis[v] = true;
                        q.push(v);
                    }
                }
            }
        }
    }
    array<int, 2> flow(int s, int t)
    {
        int f = 0;
        int cost = 0;
        h.assign(n, 0);
        spfa(s);
        while (dij(s, t))
        {
```

```
            for (int i = 0; i < n; i++)
            {
                h[i] += dis[i];
            }
            int nf = inf;
            for (int i = t; i != s; i = e[pre[i] ^ 1].to)
                nf = min(nf, e[pre[i]].cap);
            for (int i = t; i != s; i = e[pre[i] ^ 1].to)
                e[pre[i]].cap -= nf, e[pre[i] ^ 1].cap += nf;
            f += nf;
            cost += nf * h[t];
        }
        return array<int, 2>({f, cost});
    }
};
```

圆方树 (Block-Cut Tree)

定义

圆方树是针对无向图构建的一种特殊树结构。原图中的每个节点称为**圆点**，原图中的每个点双连通分量（BCC）对应圆方树中的一个**方点**。每个方点与其对应的点双连通分量中的所有圆点相连。

性质

- 圆方树中圆点仅与方点相连，方点仅与圆点相连，形成一棵二分树。
- 原图中的一个点 u 是割点，当且仅当它在圆方树中的**度数 > 1**。
- **两点间必经点**：原图中两点 u, v 之间所有的**割点**，都在圆方树中 u 到 v 的唯一简单路径上。
- **路径集合关系**：原图中 u 到 v 的所有简单路径的**并集**，恰好等于圆方树中 u, v 路径上所有**方点**所代表的点双连通分量的并集。

强连通分量

Tarjan 算法实现

使用 DFS 过程中产生的 `dfn` (时间戳) 和 `low` (追溯值) 来判定强连通分量。

复杂度

- **时间复杂度**: $O(V + E)$
- **空间复杂度**: $O(V + E)$

代码

```
struct SCC
{
    int n;
    vector<vector<int>> adj;
    vector<int> stk;
    vector<int> dfn, low, bel;
    int cur, cnt;
    SCC() {};
    SCC(int n)
    {
        init(n);
    }
    void init(int n)
    {
        this->n = n;
        adj.assign(n, {});
        dfn.assign(n, -1);
        low.resize(n);
        bel.assign(n, -1);
        cur = cnt = 0;
    }
    void add(int u, int v)
    {
        adj[u].push_back(v);
    }
    void dfs(int u)
    {
        dfn[u] = low[u] = cur++;
        stk.push_back(u);
        for (auto v : adj[u])
        {
            if (dfn[v] == -1)
            {
                dfs(v);
                low[u] = min(low[v], low[u]);
            }
            else if (bel[v] == -1)
            {
                low[u] = min(low[u], dfn[v]);
            }
        }
        if (dfn[u] == low[u])
        {
            int v;
            do
            {
                v = stk.back();
                stk.pop_back();
                bel[v] = cnt;
            } while (v != u);
            cnt++;
        }
    }
    vector<int> work()
    {
        for (int i = 0; i < n; i++)
        {
            if (dfn[i] == -1)
                dfs(i);
        }
        return bel;
    }
    vector<vector<int>> get_DAG()
    {
        vector nadj(cnt, vector<int>{});
        for (int u = 0; u < n; u++)
        {
            for (auto v : adj[u])
            {
                if (bel[u] != bel[v])
                    nadj[bel[u]].push_back(bel[v]);
            }
        }
        for (int i = 0; i < cnt; i++)
        {
            ranges::sort(nadj[i]);
            nadj[i].erase(unique(nadj[i].begin(), nadj[i].end()), nadj[i].end());
        }
        return nadj;
    }
};
```

点双连通分量 (V-BCC)

定义

在一个无向图中，若删除任意一个节点后，原图的某个子图仍然连通，则称该子图为点双连通的。每个极大点双连通子图称为点双连通分量。
特性：

1. 两个点双连通分量之间最多只有一个公共点（割点）。
2. 每一个 V-BCC 都可以看作是圆方树中的一个方点。

实现

使用 Tarjan 算法维护 `dfn` 和 `low` 数组。当满足 `low[v] >= dfn[u]` 时，说明找到了一个以 u 为根的点双。

代码

```
struct VBCC
{
    int n;
    vector<vector<int>> adj, bcc;
    vector<int> dfn, low;
    vector<int> stk;
    vector<int> cut;
    int cur;
    VBCC() {};
    void init(int n)
    {
        this->n = n;
        adj.assign(n, {});
        dfn.assign(n, -1);
        low.assign(n, -1);
        cut.assign(n, 0);
        stk.clear();
        cur = 0;
    }
    VBCC(int n)
    {
        init(n);
    }
    void add(int u, int v)
    {
        adj[u].push_back(v), adj[v].push_back(u);
    }
    void dfs(int u, int f)
    {
        dfn[u] = low[u] = cur++;
        stk.push_back(u);
        int son = 0;
        for (auto v : adj[u])
        {
            if (v == f)
                continue;
            if (dfn[v] == -1)
            {
                son++;
                dfs(v, u);
                low[u] = min(low[u], low[v]);
                if (low[v] >= dfn[u])
                {
                    if (f == -1 || son > 1)
                        cut[u] = 1;
                    bcc.push_back({});
                    int y;
                    do
                    {
                        y = stk.back();
                        bcc.back().push_back(y);
                        stk.pop_back();
                    } while (y != v);
                    bcc.back().push_back(u);
                }
            }
            else
                low[u] = min(dfn[v], low[u]);
        }
        if (f == -1 && son == 0)
        {
            bcc.push_back({});
            bcc.back().push_back(u);
        }
    }
    vector<vector<int>> work()
    {
        for (int i = 0; i < n; i++)
        {
            if (dfn[i] == -1)
            {
                dfs(i, -1);
            }
        }
        return bcc;
    }
    vector<vector<int>> get_tree()
    {
        vector t(n + bcc.size(), vector<int>());
        for (int i = 0; i < bcc.size(); i++)
        {
            int u = n + i;
            for (auto v : bcc[i])
            {
                t[v].push_back(u);
                t[u].push_back(v);
            }
        }
        return t;
    }
};
```

存边

```
struct VBCC
{
    int n;
    vector<vector<array<int, 2>>> adj;
    vector<vector<int>> bcc;
    vector<int> dfn, low;
    vector<int> stk;
    vector<int> cut;
    vector<array<int, 2>> e;
    vector<int> vis;
    vector<int> bel;
    int cur, cnt;
    VBCC() {};
    void init(int n)
    {
        this->n = n;
        adj.assign(n, {});
        dfn.assign(n, -1);
        low.assign(n, -1);
        cut.assign(n, 0);
        stk.clear();
        cur = 0;
        cnt = 0;
    }
    VBCC(int n, int m)
    {
        vis.assign(m, 0);
        bel.assign(m, 0);
        init(n);
    }
    void add(int u, int v)
    {
        adj[u].push_back({v, (int)e.size()}), adj[v].push_back({u, (int)e.size()});
        e.push_back({u, v});
    }
    void dfs(int u, int f)
    {
        dfn[u] = low[u] = cur++;
        for (auto [v, idx] : adj[u])
        {
            if (vis[idx])
                continue;
            vis[idx] = 1;
            stk.push_back(idx);
            if (dfn[v] == -1)
            {
                dfs(v, u);
                low[u] = min(low[u], low[v]);
                if (low[v] >= dfn[u])
                {
                    int y;
                    do
                    {
                        y = stk.back();
                        bel[y] = cnt;
                        stk.pop_back();
                    } while (y != idx);
                    cnt++;
                }
            }
            else
                low[u] = min(dfn[v], low[u]);
        }
    }
    vector<int> work()
    {
        for (int i = 0; i < n; i++)
        {
            if (dfn[i] == -1)
            {
                dfs(i, -1);
            }
        }
        return bel;
    }
    vector<vector<int>> get_tree()
    {
        vector t(n + bcc.size(), vector<int>());
        for (int i = 0; i < bcc.size(); i++)
        {
            int u = n + i;
            for (auto v : bcc[i])
            {
                t[v].push_back(u);
                t[u].push_back(v);
            }
        }
        return t;
    }
};
```

边双连通分量 (e-BCC)

Tarjan 算法实现

通过 Tarjan 算法找桥，并第二次 DFS 染色或使用栈来提取每一个边双连通分量。

代码

```
struct EBCC
{
    int n;
    vector<vector<array<int, 2>>> adj;
    vector<array<int, 2>> e;
    vector<int> dfn, low, bel, siz;
    vector<int> brg;
    vector<int> stk;
    int cur, cnt;
    EBCC() {};
    EBCC(int n)
    {
        init(n);
    }
    void init(int n)
    {
        this->n = n;
        adj.assign(n, {});
        dfn.assign(n, -1);
        low.assign(n, -1);
        bel.assign(n, -1);
        e.clear();
        stk.clear();
        siz.clear();
        cur = cnt = 0;
    };
    void add(int u, int v)
    {
        adj[u].push_back({v, (int)e.size()});
        adj[v].push_back({u, (int)e.size()});
        e.push_back({u, v});
    }
    void dfs(int u, int fa)
    {
        dfn[u] = low[u] = cur++;
        stk.push_back(u);
        for (auto [v, pos] : adj[u])
        {
            if (pos == fa)
                continue;
            if (dfn[v] == -1)
            {
                dfs(v, pos);
                low[u] = min(low[u], low[v]);
                if (low[v] > dfn[u])
                    brg.push_back(pos);
            }
            else if (bel[v] == -1 && dfn[v] < dfn[u])
            {
                low[u] = min(low[u], dfn[v]);
            }
        }
        if (dfn[u] == low[u])
        {
            int v;
            int sz = 0;
            do
            {
                v = stk.back();
                stk.pop_back();
                sz++;
                bel[v] = cnt;
            } while (v != u);
            siz.push_back(sz);
            cnt++;
        }
    }
    vector<int> work()
    {
        for (int i = 0; i < n; i++)
        {
            if (dfn[i] == -1)
            {
                dfs(i, -1);
            }
        }
        return bel;
    }
    vector<array<int, 2>> compress()
    {
        vector<array<int, 2>> edges;
        for (int i = 0; i < e.size(); i++)
        {
            auto [u, v] = e[i];
            if (bel[u] != bel[v])
                edges.push_back({bel[u], bel[v]});
        }
        return edges;
    }
};
```

AC 自动机

定义

AC 自动机的拓扑排序优化版本。在标准 AC 自动机中，如果每次文本串匹配到一个节点都暴力向上跳 `fail` 指针更新答案，会被特殊构造的数据（如全 `a` 串）卡成 $O(|S| \times \text{模式串总长})$。拓扑排序优化通过只在匹配时给当前节点打上标记，最后根据 `fail` 树的拓扑序统一向上合并贡献，将时间复杂度严格保证为 $O(|S| + \sum |T|)$。

实现

代码

```
struct AC
{
    static const int ALPHABET = 26;
    struct Node
    {
        int len, link;
        array<int, ALPHABET> nxt;
        Node() : len(0), link(0), nxt{} {};
    };
    vector<Node> t;
    AC()
    {
        init();
    }
    void init()
    {
        t.assign(2, Node());
        t[0].nxt.fill(1);
        t[0].len = -1;
    }
    int newNode()
    {
        t.emplace_back();
        return t.size() - 1;
    }
    int insert(string &a)
    {
        int p = 1;
        for (auto c : a)
        {
            int u = c - 'a';
            if (next(p, u) == 0)
            {
                t[p].nxt[u] = newNode();
                t[next(p,u)].len = t[p].len + 1;
            }
            p =next(p,u);
        }
        return p;
    }
    void work()
    {
        queue<int> q;
        q.push(1);
        while (q.size())
        {
            int x = q.front();
            q.pop();
            for (int i = 0; i < ALPHABET; i++)
            {
                if (next(x, i) == 0)
                {
                    t[x].nxt[i] = next(link(x),i);
                }
                else
                {
                    t[next(x,i)].link = next(link(x),i);
                    q.push(next(x,i));
                }
            }
        }
    }
    int next(int p, int x)
    {
        return t[p].nxt[x];
    }
    int link(int p)
    {
        return t[p].link;
    }
    int len(int p)
    {
        return t[p].len;
    }
    int size()
    {
        return t.size();
    }
};
```

KMP 算法

实现 (结构体封装版)

KMP 算法用于在主串中快速查找模式串。其核心在于利用已匹配的信息，通过 `next` 数组（失配函数）跳过不必要的比较。

代码

```
//f:s[0,i)中匹配的长度
vector<int> kmp(string s) {
    int n = s.size();
    std::vector<int> f(n + 1);
    for (int i = 1, j = 0; i < n; i++) {
        while (j && s[i] != s[j]) {
            j = f[j];
        }
        j += (s[i] == s[j]);
        f[i + 1] = j;
    }
    return f;
}

vector<int> match(string s, string p)
{
    int n = s.size(), m = p.size();
    auto f = kmp(p);
    vector<int> pos;
    for (int i = 0, j = 0; i < n; i++)
    {
        while (j && s[i] != p[j])
            j = f[j];

        if (s[i] == p[j])
            j++;

        if (j == m)
        {
            pos.push_back(i - m + 1);
            j = f[j];
        }
    }
    return pos;
}
```

Manacher

定义

马拉车算法（Manacher's Algorithm）用于在线性 $O(n)$ 时间复杂度内，求出字符串中以每个字符（或字符间隙）为中心的最长回文子串长度。

实现

通过在原字符串的每个字符左右及首尾插入特殊字符（如 `#`），将奇数长度和偶数长度的回文串统一处理为奇数长度。同时在最首尾两端加入不同的边界符（如 `$` 和 `^`）以避免边界越界检查。在匹配过程中，利用前面已经匹配过的回文对称信息来加速当前的匹配。

代码

```
struct Manacher
{
    int n;
    string s;
    vector<int> r;
    Manacher(string _s)
    {
        s = '#';
        for (auto c : _s)
        {
            s += c, s += '#';
        }
        n = s.size();
        r.resize(n);
        init();
    }
    void init()
    {
        for (int i = 0, c = 0; i < n; i++)
        {
            if (2 * c - i >= 0 && c + r[c] > i)
            {
                r[i] = min(r[2 * c - i], c + r[c] - i);
            }
            while (i - r[i] >= 0 && i + r[i] < n && s[i - r[i]] == s[i + r[i]])
            {
                r[i]++;
            }
            if (i + r[i] > c + r[c])
                c = i;
        }
    }
    int get_max()
    {
        int res = 0;
        for (int l : r)
        {
            res = max(res, l - 1);
        }
        return res;
    }
    bool check(int x, int y)
    {
        int nl = x * 2 + 1;
        int nr = y * 2 + 1;
        int mid = nl + nr >> 1;
        return r[mid] - 1 >= y - x + 1;
    }
};
```

Trie 树

定义

Trie 树，又称字典树、单词查找树或前缀树，是一种用于快速检索的多叉树结构。 它的核心思想是**利用字符串的公共前缀来降低查询时间的开销**以达到提高效率的目的（空间换时间）。

时间复杂度：插入、查询和删除操作的时间复杂度均为 $O(|S|)$，其中 $|S|$ 为目标字符串的长度，与树中存储的字符串总数无关。

实现

采用 `vector` 动态开点的方式模拟指针，避免了内存泄漏和繁琐的指针操作，非常适合算法竞赛和日常刷题。 * `nxt`： 二维数组（或由 `vector<array>` 嵌套构成）， `nxt[p][x]` 表示节点 `p` 沿着字符 `x` 走到的下一个节点的编号。 * `cnt`： 记录以当前节点为**结尾**的单词数量。 * `pass`： 记录**经过**当前节点的单词数量（用于统计前缀数量、动态删除和查询字典序第 K 小）。

代码

```
#include<bits/stdc++.h>
using namespace std;

struct Trie{
    vector<array<int,26>>nxt;
    vector<int> cnt,pass;
    Trie(){
        init();
    };
    void init(){
        nxt.assign(1,{});
        cnt.assign(1, 0);
        pass.assign(1,0);
    }
    void insert(const string s){
        int p=0;
        for(auto c:s){
            int x=c-'a';
            if(nxt[p][x]==0){
                nxt[p][x] = nxt.size();
                nxt.emplace_back();
                cnt.emplace_back(0);
                pass.emplace_back(0);
            }
            pass[p]++;
            p = nxt[p][x];
        }
        cnt[p]++;
        pass[p]++;
    }
    int query(const string s){
        int p=0;
        for(auto c:s){
            int x=c-'a';
            if(nxt[p][x]==0)
                return 0;
            p = nxt[p][x];
        }
        return cnt[p];
    }
    int query_pre(const string s){
        int p=0;
        for(auto c:s){
            int x=c-'a';
            if(nxt[p][x]==0)return 0;
            p=nxt[p][x];
        }
        return pass[p];
    }
    bool erase(const string s){
        if(query(s)==0)return 0;
        int p=0;
        pass[p]--;
        for(auto c:s){
            int x=c-'a';
            p=nxt[p][x];
            pass[p]--;
        }
        cnt[p]--;
        return 1;
    }
    string kth(int k) {
        if (k > pass[0]) return "";
        int p = 0;
        string res = "";
        while (k > 0) {
            if (cnt[p] >= k) return res;
            k -= cnt[p];
            for (int i = 0; i < 26; ++i) {
                if (nxt[p][i]) {
                    if (k <= pass[nxt[p][i]]) {
                        res += (char)('a' + i);
                        p = nxt[p][i];
                        break;
                    } else {
                        k -= pass[nxt[p][i]];
                    }
                }
            }
        }
        return res;
    }
};
```

01-Trie

01-Trie 是一种特殊的字典树，用于维护异或和、位运算相关问题。通常处理整数的二进制位（如从高位到低位 $2^{30} \to 2^0$）。

基础实现

适用于处理固定位数的整数（如 2^{30} 范围），支持插入、删除和查询最大异或值。

代码

```
struct Trie01 {
    vector<array<int, 2>> nxt;
    vector<int> cnt, pass;
    int L; // 最大位数

    Trie01(int max_bit = 30) : L(max_bit) {
        init();
    };

    void init() {
        nxt.assign(1, {-1, -1}); // 使用-1表示空节点
        cnt.assign(1, 0);
        pass.assign(1, 0);
    }

    void insert(int val) {
        int p = 0;
        for (int i = L; i >= 0; --i) {
            int x = (val >> i) & 1;
            if (nxt[p][x] == -1) {
                nxt[p][x] = nxt.size();
                nxt.push_back({-1, -1});
                cnt.push_back(0);
                pass.push_back(0);
            }
            pass[p]++;
            p = nxt[p][x];
        }
        cnt[p]++;
        pass[p]++;
    }

    bool erase(int val) {
        if (query_count(val) == 0) return false;
        int p = 0;
        pass[p]--;
        for (int i = L; i >= 0; --i) {
            int x = (val >> i) & 1;
            p = nxt[p][x];
            pass[p]--;
        }
        cnt[p]--;
        return true;
    }

    int query_count(int val) {
        int p = 0;
        for (int i = L; i >= 0; --i) {
            int x = (val >> i) & 1;
            if (nxt[p][x] == -1) return 0;
            p = nxt[p][x];
        }
        return cnt[p];
    }

    // 查询与 val 异或能得到的最大值
    int query_max_xor(int val) {
        if (pass[0] == 0) return 0;
        int p = 0, res = 0;
        for (int i = L; i >= 0; --i) {
            int x = (val >> i) & 1;
            // 尽量走相反的路来保证该位异或为1
            if (nxt[p][x ^ 1] != -1 && pass[nxt[p][x ^ 1]] > 0) {
                res |= (1 << i);
                p = nxt[p][x ^ 1];
            } else {
                p = nxt[p][x];
            }
        }
        return res;
    }

    // 类似原模板的kth，求排名第k小的数值（字典序/数值大小）
    int kth(int k) {
        if (k > pass[0]) return -1;
        int p = 0, res = 0;
        for (int i = L; i >= 0; --i) {
            // 在这一层，如果节点有停止计数(cnt)，逻辑会略有不同
            // 01Trie通常只在叶子层计数，所以这里逻辑简化为按位分流
            int left_child = nxt[p][0];
            int left_pass = (left_child != -1) ? pass[left_child] : 0;

            if (k <= left_pass) {
                p = left_child;
            } else {
                k -= left_pass;
                res |= (1 << i);
                p = nxt[p][1];
            }
        }
        return res;
```

```
    }
};
```

后缀数组（SA）

定义

后缀数组将非空字符串的所有后缀按字典序排列，可用于处理子串排序、最长重复子串和最长公共子串等问题。该实现使用倍增法构造后缀数组，并使用 Kasai 算法求相邻后缀的最长公共前缀，时间复杂度为 $O(n \log n)$，空间复杂度为 $O(n)$。

实现

所有数组均从 0 开始，下标和数组值都是原字符串中的位置：

- `sa[i]`：字典序第 `i` 小的后缀在原字符串中的起点。
- `rk[i]`：起点为 `i` 的后缀在 `sa` 中的排名，即 `sa[rk[i]] = i`。
- `lc[i]`：相邻后缀 `sa[i]` 与 `sa[i + 1]` 的最长公共前缀长度，因此其长度为 `n - 1`。

倍增过程中按第二关键字和第一关键字进行基数排序，每轮将已比较的前缀长度翻倍。代码要求字符串非空，并使用 C++23 的 `ranges::iota`。

代码

```
struct SA
{
    int n;
    vector<int> sa, rk, lc;
    SA(string &s) : n(s.size()), sa(s.size()), rk(s.size()), lc(s.size() - 1)
    {
        ranges::iota(sa, 0);
        ranges::sort(sa, [&](int a, int b)
                     { return s[a] < s[b]; });
        rk[sa[0]] = 0;
        for (int i = 1; i < n; i++)
        {
            rk[sa[i]] = rk[sa[i - 1]] + (s[sa[i - 1]] != s[sa[i]]);
        }
        int k = 1;
        vector<int> tmp, cnt(n);
        tmp.reserve(n);
        while (rk[sa[n - 1]] < n - 1)
        {
            tmp.clear();
            for (int i = 0; i < k; i++)
            {
                tmp.push_back(n - k + i);
            }
            for (auto i : sa)
            {
                if (i >= k)
                    tmp.push_back(i - k);
            }
            ranges::fill(cnt, 0);
            for (int i = 0; i < n; i++)
                cnt[rk[i]]++;
            for (int i = 1; i < n; i++)
                cnt[i] += cnt[i - 1];
            for (int i = n - 1; i >= 0; i--)
            {
                sa[--cnt[rk[tmp[i]]]] = tmp[i];
            }
            swap(rk, tmp);
            rk[sa[0]] = 0;
            for (int i = 1; i < n; i++)
            {
                rk[sa[i]] = rk[sa[i - 1]] + (tmp[sa[i - 1]] < tmp[sa[i]] || sa[i - 1] + k == n || tmp[sa[i - 1] + k] < tmp[sa[i] + k]);
            }
            k <<= 1;
        }
        for (int i = 0, j = 0; i < n; i++)
        {
            if (rk[i] == 0)
                j = 0;
            else
            {
                for (j -= j > 0; i + j < n && sa[rk[i] - 1] + j < n && s[i + j] == s[sa[rk[i] - 1] + j];)
                {
                    j++;
                }
                lc[rk[i] - 1] = j;
            }
        }
    }
};
```

后缀自动机（SAM）

定义

后缀自动机（Suffix Automaton, 简称 SAM）是一个能识别一个字符串的所有子串的确定性有限状态自动机（DFA）。它的状态数和转移数均是线性的（节点数不超过 $2n-1$，转移数不超过 $3n-2$），常用于解决复杂的字符串子串匹配、出现次数统计、最长公共子串等问题。

实现

后缀自动机的线性时间在线构造算法。采用结构体封装，节点下标从 1 开始，0 作为空指针，字符集默认为小写字母。

代码

```
struct SAM
{
    static const int ALPHABET = 26;
    struct Node
    {
        int len, link;
        array<int, ALPHABET> nxt;
        Node() : len{}, link{}, nxt{} {};
    };
    vector<Node> t;
    SAM() { init(); };
    void init()
    {
        t.assign(2, Node());
        t[0].nxt.fill(1);
        t[0].len = -1;
    }
    int newNode()
    {
        t.emplace_back();
        return t.size() - 1;
    }
    int insert(int p, int c)
    {
        if (next(p, c))
        {
            int q = next(p, c);
            if (t[q].len == t[p].len + 1)
                return q;
            int r = newNode();
            t[r].len = t[p].len + 1;
            t[r].link = link(q);
            t[r].nxt = t[q].nxt;
            t[q].link = r;
            while (t[p].nxt[c] == q)
            {
                t[p].nxt[c] = r;
                p = link(p);
            }
            return r;
        }
        int cur = newNode();
        t[cur].len = t[p].len + 1;
        while (!next(p, c))
        {
            t[p].nxt[c] = cur;
            p = link(p);
        }
        t[cur].link = insert(p, c);
        return cur;
    }
    int next(int p, int x)
    {
        return t[p].nxt[x];
    }
    int link(int p)
    {
        return t[p].link;
    }
    int len(int p)
    {
        return t[p].len;
    }
    int size()
    {
        return t.size();
    }
};
```

字符串哈希 (String Hash)

实现 (双哈希结构体封装)

使用两个大质数（如 10^9+7 和 10^9+9）进行双哈希，基数通常选取 131 或 13331。通过前缀和思想实现 $O(1)$ 复杂度提取子串哈希值。

代码

```cpp
#include <bits/stdc++.h>

using i64 = long long;

struct Hash {
    static constexpr int M1 = 1e9 + 7, M2 = 1e9 + 9;
    static constexpr int B = 131;
    std::vector<int> h1, h2, p1, p2;

    Hash(const std::string& s) {
        int n = s.size();
        h1.resize(n + 1);
        h2.resize(n + 1);
        p1.resize(n + 1);
        p2.resize(n + 1);
        p1[0] = p2[0] = 1;
        for (int i = 0; i < n; i++) {
            h1[i + 1] = (1LL * h1[i] * B + s[i]) % M1;
            h2[i + 1] = (1LL * h2[i] * B + s[i]) % M2;
            p1[i + 1] = 1LL * p1[i] * B % M1;
            p2[i + 1] = 1LL * p2[i] * B % M2;
        }
    }

    // 返回闭区间 [l, r] 的哈希值，索引从 0 开始
    std::pair<int, int> get(int l, int r) {
        int r1 = (h1[r + 1] - 1LL * h1[l] * p1[r - l + 1] % M1 + M1) % M1;
        int r2 = (h2[r + 1] - 1LL * h2[l] * p2[r - l + 1] % M2 + M2) % M2;
        return {r1, r2};
    }
};
```


扩展 KMP（Z 函数）

扩展 KMP（在国外常被称为 Z 函数）用于在线性时间内求解：
1. 字符串 S 的所有后缀与 S 本身的最长公共前缀（LCP）长度。
2. 文本串 T 的所有后缀与模式串 S 的最长公共前缀（LCP）长度。

实现

常用于单串性质的分析，时间复杂度为 $O(n)$。

代码

```cpp
#include <bits/stdc++.h>
#define int long long
using namespace std;

// https://www.luogu.com.cn/problem/P5410
vector<int> Z(string &s)
{
     int n = s.size();
    vector<int> z(n);
    z[0] = n;
    for (int i = 1, j = 1; i < n; i++)
    {
        z[i] = max(0LL, min(j + z[j] - i, z[i - j]));
        while (i + z[i] < n && s[z[i]] == s[i + z[i]])
            z[i]++;
        if (i + z[i] > j + z[j])
            j = i;
    }
    return z;
}
void solve()
{
    string a, b;
    cin >> a >> b;
    int n = a.size(), m = b.size();
    auto z = Z(b);
    vector<int> e(n);
    for (int i = 0, j = 0; i < n; i++)
    {
        e[i] = max(0LL, min(j + e[j] - i, z[i - j]));
        while (i + e[i] < n && e[i] < m && a[i + e[i]] == b[e[i]])
            e[i]++;
        if (i + e[i] > j + e[j])
            j = i;
    }
    int valz = 0;
    for (int i = 0; i < m; i++)
    {
        valz ^= (i + 1) * (z[i] + 1);
    }
    int vale = 0;
    for (int i = 0; i < n; i++)
```

```
    {
        vale ^= (i + 1) * (e[i] + 1);
    }
    cout << valz << '\n'
         << vale << '\n';
}
signed main()
{
    ios::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

ST 表

定义

ST 表用于在不可修改的序列上查询区间最小值。数组采用 0 下标， `query(l, r)` 查询非空半开区间 `[l, r)`；预处理时间和空间复杂度均为 $O(n \log n)$，单次查询时间复杂度为 $O(1)$。

实现

`st[k][i]` 维护区间 $[i, i + 2^k)$ 的最小值。查询时令 $k = \lfloor \log_2(r - l) \rfloor$，合并分别贴近左右端点的两个长度为 2^k 的区间；由于最小值运算满足幂等性，两个区间重叠不会影响答案。

代码

```
struct ST
{
    int n, K;
    vector<vector<int>> st;
    vector<int> lg;
    ST() {}
    ST(int n)
    {
        vector<int> a(n, inf);
        init(a);
    }
    ST(vector<int> &a)
    {
        init(a);
    }
    void init(vector<int> &a)
    {
        n = a.size();
        lg.resize(n + 1);
        for (int i = 2; i <= n; i++)
            lg[i] = lg[i >> 1] + 1;
        int K = lg[n] + 1;
        st.assign(K, vector<int>(n));
        for (int i = 0; i < n; i++)
        {
            st[0][i] = a[i];
        }
        for (int k = 1; k < K; k++)
        {
            for (int i = 0; i + (1LL << k) <= n; i++)
            {
                st[k][i] = min(st[k - 1][i],
                               st[k - 1][i + (1LL << (k - 1))]);
            }
        }
    }
    int query(int l, int r)
    {
        int k = lg[r - l];
        return min(st[k][l], st[k][r - (1LL << k)]);
    }
};
```

主席树

定义

主席树全称是**可持久化权值线段树**。它的核心任务是：每插入一个数，都要保留一个节点的线段树版本。
可以实现区间第k小查询。

实现

- **暴力想法**：每插入一个数就新开一棵动态开点线段树。N 个数就是 $N \times (N \log V)$，内存瞬间爆炸。
- **复用节点**： 当你修改一个节点时，从根到叶子只有 $\log V$ 个节点会变。**剩下的子树和前一个版本一模一样！** 我们只需要新建这 $\log V$ 个节点，让新节点的左右指针指向**前一个版本**中没变过的儿子节点即可。

代码

```
#include <bits/stdc++.h>
using namespace std;

struct SegTree
{
    struct Node
    {
        int l;
        int r;
        int cnt;
    };
    vector<int> root;
    int tot;
    int n; // 值域
    vector<Node> info;
    SegTree(int n, int nodes, int ver) : n(n), tot(0)
    {
        info.assign(nodes, {0, 0, 0});
        root.assign(ver + 1, 0);
    }
    void pull(int p)
    {
        int l = info[p].l, r = info[p].r;
        info[p].cnt = info[l].cnt + info[r].cnt;
    }

    void modify(int &p, int pre, int l, int r, int x, int v)
    {
        p = ++tot;
        info[p] = info[pre];

        if (r - l == 1)
        {
            info[p].cnt += v;
            return;
        }
        int mid = l + r >> 1;
        if (x < mid)
        {
            modify(info[p].l, info[p].l, l, mid, x, v);
        }
        else
        {
            modify(info[p].r, info[p].r, mid, r, x, v);
        }
        pull(p);
    }
    void modify(int ver, int pre_ver, int x, int v = 1)
    {
        modify(root[ver], root[pre_ver], 0, n, x, v);
    }

    int query_cnt(int L, int R, int l, int r, int x, int y)
    {
        if (l >= y || r <= x)
            return 0;
        if (x <= l && r <= y)
            return info[R].cnt - info[L].cnt;
        int mid = l + r >> 1;
        return query_cnt(info[L].l, info[R].l, l, mid, x, y) + query_cnt(info[L].r, info[R].r, mid, r, x, y);
    }
    int query_cnt(int L, int R, int l, int r)
    {
        return query_cnt(root[L], root[R], 0, n, l, r);
    }
    int query_rank(int L, int R, int x)
    {
        return query_cnt(root[L], root[R], 0, n, 0, x) + 1;
    }
    int kth(int L, int R, int l, int r, int k)
    {
        if (r - l == 1)
            return l;
        int mid = l + r >> 1;

        int cntl = info[info[R].l].cnt - info[info[L].l].cnt;

        if (k <= cntl)
        {
            return kth(info[L].l, info[R].l, l, mid, k);
        }
        else
        {
            return kth(info[L].r, info[R].r, mid, r, k - cntl);
        }
    }
    int kth(int L, int R, int k)
    {
        return kth(root[L], root[R], 0, n, k);
    }
};
```

```
void solve()
{
    int n, m;
    cin >> n >> m;
    vector<int> a(n + 1);
    for (int i = 1; i <= n; i++)
        cin >> a[i];
    SegTree T(1e9, n * 40, n);
    for (int i = 1; i <= n; i++)
    {
        T.modify(i, i - 1, a[i]);
    }
    while (m--)
    {
        int l, r, k;
        cin >> l >> r >> k;
        cout << T.kth(l - 1, r, k) << '\n';
    }
}
signed main()
{
    ios::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    int t = 1;
    // cin>>t;
    while (t--)
        solve();
    return 0;
}
```

可撤销并查集

定义

可撤销并查集支持在传统的合并（Union）和查询（Find）操作之上，撤销最近执行的合并操作，回退到之前的历史状态。由于路径压缩会破坏树的原始形态并带来大量状态变更，可撤销并查集**严禁使用路径压缩**，必须采用启发式合并（按大小 `sz` 或深度 `dep` 合并）以保证单次操作的时间复杂度稳定在 $O(\log n)$。

实现

使用 `vector<pair<int, int>>` 作为操作栈，记录每次成功合并的父子节点对 `(x, y)`。撤销时按先进后出的顺序恢复 `p` 数组和 `sz` 数组。

代码

```cpp
#include <bits/stdc++.h>
using namespace std;

struct DSU {
    vector<int> f, siz;
    int cnt;
    vector<pair<int, int>> st; // 记录历史操作的栈

    DSU() {}
    DSU(int n) { init(n); }

    void init(int n) {
        f.resize(n);
        iota(f.begin(), f.end(), 0);
        siz.assign(n, 1);
        cnt = n;
        st.clear();
    }

    int find(int x) {
        while (x != f[x]) {
            x = f[x];
        }
        return x;
    }

    bool merge(int x, int y) {
        x = find(x);
        y = find(y);
        if (x == y) return 0;

        // 启发式合并：小树合入大树，保证树高 O(log n)
        if (siz[x] < siz[y]) swap(x, y);

        st.push_back({x, y}); // 记录谁成为了谁的父亲
        siz[x] += siz[y];
        cnt--;
        f[y] = x;
        return 1;
    }

    int esize(int x) {
        return siz[find(x)];
    }

    int count() {
        return cnt;
    }

    // 获取当前时间戳
    int time() {
        return st.size();
    }

    // 回滚至指定时间戳 t
    void roll_back(int t) {
        while (st.size() > t) {
            auto [x, y] = st.back();
            st.pop_back();
            f[y] = y;         // 恢复父节点指向自己
            siz[x] -= siz[y]; // 恢复连通块大小
            cnt++;            // 恢复连通块数量
        }
    }
};
```

并查集

实现

代码

```
struct DSU
{
    vector<int> f, siz;
    int cnt;

    DSU() {}
    DSU(int n) { init(n); }

    void init(int n)
    {
        f.resize(n);
        iota(f.begin(), f.end(), 0);
        siz.assign(n, 1);
        cnt = n;
    }

    int find(int x)
    {
        return x==f[x]?f[x]:f[x]=find(f[x]);
    }

    bool merge(int x, int y)
    {
        x = find(x);
        y = find(y);
        if (x == y)
            return 0;
        siz[x] += siz[y];
        cnt--;
        f[y] = x;
        return 1;
    }

    int esize(int x)
    {
        return siz[find(x)];
    }

    int count()
    {
        return cnt;
    }
};
```

权值线段树

定义

权值线段树（Value Segment Tree）是线段树的一种变体。

- **普通线段树**的节点维护的是**数组下标**区间 $[l, r)$ 的统计信息（如区间和、最值）。
- **权值线段树**的节点维护的是**数值（即权值区间** $[l, r)$ 的统计信息，通常是该值域范围内数字出现的**频数（次数）**。

它的核心优势在于能够在 $\mathcal{O}(\log V)$（V 为值域大小）的时间复杂度内，高效解决全局第 k 小/大、查询元素排名等问题，是进阶数据结构（如可持久化线段树/主席树）的基础。

实现

权值线段树的核心操作主要基于线段树的二分性质：

1. **单点修改 (`modify`)**：
   在插入或删除某个数字 x 时，相当于在值域线段树中，将叶子节点 x 对应的频数加 1 或减 1。然后自底向上更新父节点的频数总和。
2. **查询全局第 k 小 (`kth`)**：
   - 从根节点开始，假设当前节点维护的值域为 $[l, r)$，左子树维护 $[l, mid)$，其频数和为 $left_cnt$。
   - 如果 $k \le left_cnt$，说明第 k 小的数落在左半值域，向左子树递归。
   - 如果 $k > left_cnt$，说明左半值域的数不够，第 k 小的数在右半值域。此时向右子树递归，**但要将 k 减去 $left_cnt$**（即在右半部分寻找第 $k - left_cnt$ 小的数）。
   - 当递归到叶子节点（即 $r - l == 1$）时，当前的 l 就是我们要找的数值。

如果题目的数值范围极大（如 10^9）或包含负数，直接建树会内存超限（MLE）。此时必须配合**离散化**（将大数值映射到 $1 \dots N$ 的连续排名），或者使用**动态开点**技术。

代码

以下是基于半开半闭区间 `[l, r)` 且无懒标记的权值线段树模板（适用于较小值域或已离散化后的数组）：

```cpp
#include <bits/stdc++.h>
#define int long long
using namespace std;

const int inf = 1e9;
struct Info
{
    int cnt;
    Info() : cnt(0){};
    Info(int value) :cnt(value) {};
    Info operator+(const Info &other) const
    {
        return Info(cnt+other.cnt);
    }
};
struct ValueSegTree
{
    int n;
    vector<int> tag;
    vector<Info> info;
    ValueSegTree(int _n) : n(_n), tag(4 * _n), info(4 * n) {};
    void pull(int p) // 更新上移
    {
        info[p] = info[p << 1] + info[p << 1 | 1];
    }
    // 区间查询[x,y)
    Info query(int p, int l, int r, int x, int y)
    {
        if (l >= y || r <= x)
            return {};
        if (l >= x && r <= y)
        {
            return info[p];
        }
        int mid = l + r >> 1;
        return query(p << 1, l, mid, x, y) + query(p << 1 | 1, mid, r, x, y);
    };
    int query(int x, int y)
    {
        return query(1, 0, n, x, y).cnt;
    };

    int rank(int val){
        return query(0,val)+1;
    }

    // 单点修改:在x上加v
    void modify(int p, int l, int r, int x, int v)
    {
        if (r - l == 1)
        {
            info[p].cnt += v;
            return;
        }
        int mid = l + r >> 1;
        if (x < mid)
        {
            modify(p << 1, l, mid, x, v);
        }
        else
        {
            modify(p << 1 | 1, mid, r, x, v);
        }
        pull(p);
    }
    void modify(int x, int v)
    {
        modify(1, 0, n, x, v);
    }

    int query_kth(int p,int l,int r,int k){
        if(r-l==1){
            return l;
        }
        int mid=l+r>>1;
        int l_cnt=info[p<<1].cnt;
        if(k<=l_cnt)
            return query_kth(p<<1,l,mid,k);
        else
            return query_kth(p<<1|1,mid,r,k-l_cnt);
    }
    int query_kth(int k){
        return query_kth(1,0,n,k);
    }
};
void solve()
{
    int n;
    cin>>n;
    vector<array<int,2>>q(n);
    vector<int>b;
    for(auto &[op,v]:q){
        cin>>op>>v;
```

```
        b.push_back(v);
    }
    ranges::sort(b);
    b.erase(unique(b.begin(),b.end()),b.end());
    auto get_idx=[&](int x){
        return ranges::lower_bound(b,x)-b.begin()+1;
    };
    ValueSegTree T((int)b.size()+1);

    for(auto [op,v]:q){
        int idx=get_idx(v);
        if(op==1){
            T.modify(idx,1);
        }
        else if(op==2){
            T.modify(idx,-1);
        }
        else if(op==3){
            cout<<T.rank(idx)<<'\n';
        }
        else if(op==4){
            cout<<b[T.query_kth(v)-1]<<'\n';
        }
        else if(op==5){
            cout<<b[T.query_kth(T.rank(idx)-1)-1]<<'\n';
        }
        else{
            cout<<b[T.query_kth(T.query(0,idx+1)+1)-1]<<'\n';
        }
    }
}
signed main()
{
    ios::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

动态开点权值线段树

对于普通的权值线段树，直接开 $4*n$ 的数组可能会爆内存。采取动态开点线段树就可以解决这个问题。

实现

一开始，我们只有一个空荡荡的根节点。只有当我们要修改某个位置（走过某条路径）时，如果发现前面的节点还没建出来，我们就**临时给它分配一个编号**。

代码

```
struct SegTree{
    struct Node{
        int l;
        int r;
        int cnt;
    };
    int root;
    int tot;
    int n;
    vector<Node>info;
    SegTree(int n,int cnt):n(n),root(0),tot(0){
        info.assign(cnt,{0,0,0});
        tot=0;
        root=0;
    }
    void pull(int p){
        int l=info[p].l,r=info[p].r;
        info[p].cnt=info[l].cnt+info[r].cnt;
    }

    void modify(int &p,int l,int r,int x,int v){
        if(!p)p=++tot;
        if(r-l==1){
            info[p].cnt+=v;
            return ;
        }
        int mid=l+r>>1;
        if(x<mid){
            modify(info[p].l,l,mid,x,v);
        }
        else{
            modify(info[p].r,mid,r,x,v);
        }
        pull(p);
    }
    void modify(int x,int v){
        return modify(root,0,n,x,v);
    }

    int query(int p,int l,int r,int x,int y){
        if(!p||l>=y||r<=x){
            return 0;
        }
        if(l>=x&&r<=y){
            return info[p].cnt;
        }
        int mid=l+r>>1;
        return query(info[p].l,l,mid,x,y)+query(info[p].r,mid,r,x,y);
    }
    int query(int x,int y){
        return query(root,0,n,x,y);
    }
    int rank(int val){
        return query(0,val)+1;
    }
    int kth(int p,int l,int r,int k){
        if(r-l==1)
            return l;
        int mid=l+r>>1;
        int cntl=info[info[p].l].cnt;
        int cntr=info[info[p].r].cnt;
        if(k<=cntl){
            return kth(info[p].l,l,mid,k);
        }
        else {
            return kth(info[p].r,mid,r,k-cntl);
        }
    }
    int kth(int k){
        return kth(root,0,n,k);
    }
};
```

笛卡尔树

定义

笛卡尔树是同时满足二叉搜索树性质和堆性质的二叉树：结点按下标中序排列，权值默认满足小根堆性质。下标采用 0 下标；建树的时间复杂度为 $O(n)$，空间复杂度为 $O(n)$。

实现

单调栈维护当前最右链，栈中结点的权值单调不降。插入结点 `i` 时，弹出所有权值大于 `a[i]` 的结点：最后弹出的结点成为 `i` 的左儿子，剩余栈顶成为 `i` 的父亲， `i` 同时成为该栈顶的右儿子。每个结点至多入栈、出栈一次；权值相等时不弹栈，因此下标较小的结点优先作为祖先。

代码

```cpp
#include <functional>
#include <vector>

using namespace std;

struct CartesianTree
{
    int n, root;
    vector<int> lc, rc, fa;

    CartesianTree() : n(0), root(-1) {}

    template <class T, class Compare = less<T>>
    CartesianTree(const vector<T> &a, Compare comp = Compare())
    {
        init(a, comp);
    }

    template <class T, class Compare = less<T>>
    void init(const vector<T> &a, Compare comp = Compare())
    {
        n = static_cast<int>(a.size());
        root = -1;
        lc.assign(n, -1);
        rc.assign(n, -1);
        fa.assign(n, -1);

        vector<int> stk;
        stk.reserve(n);
        for (int i = 0; i < n; i++)
        {
            int last = -1;
            while (!stk.empty() && comp(a[i], a[stk.back()]))
            {
                last = stk.back();
                stk.pop_back();
            }
            if (!stk.empty())
            {
                rc[stk.back()] = i;
                fa[i] = stk.back();
            }
            if (last != -1)
            {
                lc[i] = last;
                fa[last] = i;
            }
            stk.push_back(i);
        }
        if (!stk.empty())
            root = stk.front();
    }
};
```

默认比较器 `less<T>` 建立小根笛卡尔树；传入 `greater<T>` 可建立大根笛卡尔树。空数组的 `root` 为 `-1`，不存在的父结点或儿子也用 `-1` 表示。

线段树

带懒标记

代码

```
#include <bits/stdc++.h>
#define int long long
using namespace std;

const int inf = 1e9;
struct Info
{
    int sum, max, min;
    int size;
    Info() : sum(), max(-inf), min(inf), size(0) {};
    Info(int value) : sum(value), max(value), min(value), size(1) {};
    Info operator+(const Info &other) const
    {
        Info res;
        res.sum = sum + other.sum;
        res.max = std::max(max, other.max);
        res.min = std::min(min, other.min);
        res.size = size + other.size;
        return res;
    }
};
struct SegTree
{
    int n;
    vector<int> tag;
    vector<Info> info;
    SegTree(int _n) : n(_n), tag(4 * _n), info(4 * n) {};
    void pull(int p) // 更新上移
    {
        info[p] = info[p << 1] + info[p << 1 | 1];
    }
    void add(int p, int v)
    {
        tag[p] += v;
        info[p].sum += v * info[p].size;
        info[p].max += v;
        info[p].min += v;
    }
    void push(int p) // 懒标记下放
    {
        add(p << 1, tag[p]);
        add(p << 1 | 1, tag[p]);
        tag[p] = 0;
    }
    // 区间查询[x,y)
    Info query(int p, int l, int r, int x, int y)
    {
        if (l >= y || r <= x)
            return {};
        if (l >= x && r <= y)
        {
            return info[p];
        }
        int mid = l + r >> 1;
        push(p);
        return query(p << 1, l, mid, x, y) + query(p << 1 | 1, mid, r, x, y);
    };
    Info query(int x, int y)
    {
        return query(1, 0, n, x, y);
    };
    // 区间修改[x,y)
    void rangeAdd(int p, int l, int r, int x, int y, int v)
    {
        if (l >= y || r <= x)
            return;
        if (l >= x && r <= y)
        {
            return add(p, v);
        }
        int mid = l + r >> 1;
        push(p);
        rangeAdd(p << 1, l, mid, x, y, v);
        rangeAdd(p << 1 | 1, mid, r, x, y, v);
        pull(p);
    };
    void rangeAdd(int x, int y, int v)
    {
        rangeAdd(1, 0, n, x, y, v);
    }
    // 单点修改
    void modify(int p, int l, int r, int x, const Info &v)
    {
        if (r - l == 1)
        {
            info[p] = v;
            return;
        }
        int mid = l + r >> 1;
        push(p);
        if (x < mid)
```

```
        {
            modify(p << 1, l, mid, x, v);
        }
        else
        {
            modify(p << 1 | 1, mid, r, x, v);
        }
        pull(p);
    }
    void modify(int x, int v)
    {
        modify(1, 0, n, x, Info(v));
    }
};
void solve()
{
    int n, m;
    cin >> n >> m;
    SegTree T(n);
    for (int i = 0; i < n; i++)
    {
        int x;
        cin >> x;
        T.modify(i, x);
    }
    while (m--)
    {
        int op;
        cin >> op;
        if (op == 1)
        {
            int x, y, k;
            cin >> x >> y >> k;
            T.rangeAdd(x - 1, y, k);
        }
        else
        {
            int x, y;
            cin >> x >> y;
            Info v = T.query(x - 1, y);
            cout << v.sum << '\n';
        }
    }
}
signed main()
{
    ios::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

代码

不带懒标记

```cpp
#include <bits/stdc++.h>
#define int long long
using namespace std;

const int inf = 1e9;
struct Info
{
    int sum, max, min;
    int size;
    Info() : sum(), max(-inf), min(inf), size(0) {};
    Info(int value) : sum(value), max(value), min(value), size(1) {};
    Info operator+(const Info &other) const
    {
        Info res;
        res.sum = sum + other.sum;
        res.max = std::max(max, other.max);
        res.min = std::min(min, other.min);
        res.size = size + other.size;
        return res;
    }
};
struct SegTree
{
    int n;
    vector<Info> info;
    SegTree(int _n) : n(_n), info(4 * _n) {};
    void pull(int p) // 更新上移
    {
        info[p] = info[p << 1] + info[p << 1 | 1];
    }
    // 区间查询[x,y)
    Info query(int p, int l, int r, int x, int y)
    {
        if (l >= y || r <= x)
            return {};
        if (l >= x && r <= y)
        {
            return info[p];
        }
        int mid = l + r >> 1;
        return query(p << 1, l, mid, x, y) + query(p << 1 | 1, mid, r, x, y);
    };
    Info query(int x, int y)
    {
        return query(1, 0, n, x, y);
    };
    // 单点修改
    void modify(int p, int l, int r, int x, const Info &v)
    {
        if (r - l == 1)
        {
            info[p] = v;
            return;
        }
        int mid = l + r >> 1;
        if (x < mid)
        {
            modify(p << 1, l, mid, x, v);
        }
        else
        {
            modify(p << 1 | 1, mid, r, x, v);
        }
        pull(p);
    }
    void modify(int x, int v)
    {
        modify(1, 0, n, x, Info(v));
    }
};
void solve()
{
    int n, m;
    cin >> n >> m;
    SegTree T(n);
    for (int i = 0; i < n; i++)
    {
        int x;
        cin >> x;
        T.modify(i, x);
    }
    while (m--)
    {
        int op;
        cin >> op;
        if (op == 1)
        {
            int x, y, k;
            cin >> x >> y >> k;
            T.rangeAdd(x - 1, y, k);
        }
        else
        {
```

```
            int x, y;
            cin >> x >> y;
            Info v = T.query(x - 1, y);
            cout << v.sum << '\n';
        }
    }
}
signed main()
{
    ios::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

线段树分治

定义

线段树分治是一种基于时间轴的离线算法。当题目中元素或操作具有“生存时间区间”时，将时间轴作为线段树的区间，将每个元素的存活期转化为线段树上的区间覆盖。随后通过 DFS 遍历线段树，配合**可撤销**的数据结构（如可撤销并查集）维护状态。当遍历至叶子节点时处理该特定时刻的询问，回溯时严格撤销状态变化。

实现

线段树分治 + 可撤销并查集（最基础且最常见的搭配）。
区间按 `[l, r)` 左闭右开规范处理。

代码

```
#include <bits/stdc++.h>
using namespace std;

struct DSU
{
    vector<int> f, siz;
    int cnt;
    vector<pair<int, int>> st; // 记录历史操作的栈

    DSU() {}
    DSU(int n) { init(n); }

    void init(int n)
    {
        f.resize(n);
        iota(f.begin(), f.end(), 0);
        siz.assign(n, 1);
        cnt = n;
        st.clear();
    }

    int find(int x)
    {
        while (x != f[x])
        {
            x = f[x];
        }
        return x;
    }

    bool merge(int x, int y)
    {
        x = find(x);
        y = find(y);
        if (x == y)
            return 0;

        // 启发式合并：小树合入大树，保证树高 O(log n)
        if (siz[x] < siz[y])
            swap(x, y);

        st.push_back({x, y}); // 记录谁成为了谁的父亲
        siz[x] += siz[y];
        cnt--;
        f[y] = x;
        return 1;
    }

    int esize(int x)
    {
        return siz[find(x)];
    }

    int count()
    {
        return cnt;
    }

    // 获取当前时间戳
    int time()
    {
        return st.size();
    }

    // 回滚至指定时间戳 t
    void roll_back(int t)
    {
        while (st.size() > t)
        {
            auto [x, y] = st.back();
            st.pop_back();
            f[y] = y;         // 恢复父节点指向自己
            siz[x] -= siz[y]; // 恢复连通块大小
            cnt++;            // 恢复连通块数量
        }
    }
};
struct Info
{
    array<int, 2> e;
    Info() {};
    Info(int u, int v) { e = {u, v}; };
    Info(array<int, 2> e) : e(e) {};
};
struct SegTree
{
    int n;
    vector<vector<Info>> info;

    SegTree(int n) : n(n), info(4 * n) {};
    void modify(int p, int l, int r, int ql, int qr, Info e)
```

```
    {
        if (ql <= l && r <= qr)
        {
            info[p].push_back(e);
            return;
        }
        if (ql >= r || qr <= l)
            return;
        int mid = l + r >> 1;
        if (ql < mid)
            modify(p << 1, l, mid, ql, qr, e);
        if (qr > mid)
            modify(p << 1 | 1, mid, r, ql, qr, e);
    }
    void modify(int l, int r, array<int, 2> e)
    {
        modify(1, 0, n, l, r, e);
    }
};
void solve()
{
    ios::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    int n, m, k;
    cin >> n >> m >> k;
    SegTree T(k);
    for (int i = 1; i <= m; i++)
    {
        int u, v, l, r;
        cin >> u >> v >> l >> r;
        T.modify(l, r, {u, v});
    }
    DSU dsu(2 * n + 1);

    auto dfs = [&](auto &&self, int p, int l, int r, bool ok) -> void
    {
        int cur = dsu.time();
        if (ok)
        {
            for (auto e : T.info[p])
            {
                auto [u, v] = e.e;
                if (dsu.find(u) == dsu.find(v))
                {
                    ok = 0;
                    break;
                }
                dsu.merge(u, v + n);
                dsu.merge(v, u + n);
            }
        }
        if (r - l == 1)
        {
            cout << (ok ? "Yes\n" : "No\n");
        }
        else
        {
            int mid = l + r >> 1;
            self(self, p << 1, l, mid, ok);
            self(self, p << 1 | 1, mid, r, ok);
        }
        dsu.roll_back(cur);
    };
    dfs(dfs, 1, 0, k, 1);
}
signed main()
{
    solve();
    return 0;
}
```

线段树合并与分裂 (Segment Tree Merge & Split)

定义

线段树合并用于将两棵动态开点线段树的信息整合到一起。线段树分裂则是合并的逆过程，将一棵线段树中指定范围的结点拆分出来形成一棵新树。

复杂度

- **合并**: 均摊时间复杂度为 $O(\log n)$，其中 n 为值域范围。总复杂度通常取决于插入的总结点数。
- **分裂**: 每次分裂的时间复杂度为 $O(\log n)$。
- **空间**: 动态开点通常需要 $O(m \log n)$ 的空间，m 为操作次数。

实现

代码

```
struct SegTree
{
    struct Node
    {
        int l;
        int r;
        int cnt;
    };
    vector<int> root;
    int tot;
    int n; // 值域
    vector<Node> info;
    SegTree(int n, int nodes, int ver) : n(n), tot(0)
    {
        info.assign(nodes, {0, 0, 0});
        root.assign(ver + 1, 0);
    }
    void pull(int p)
    {
        int l = info[p].l, r = info[p].r;
        info[p].cnt = info[l].cnt + info[r].cnt;
    }

    void modify(int &p, int pre, int l, int r, int x, int v)
    {
        p = ++tot;
        info[p] = info[pre];

        if (r - l == 1)
        {
            info[p].cnt += v;
            return;
        }
        int mid = l + r >> 1;
        if (x < mid)
        {
            modify(info[p].l, info[p].l, l, mid, x, v);
        }
        else
        {
            modify(info[p].r, info[p].r, mid, r, x, v);
        }
        pull(p);
    }
    void modify(int ver, int pre_ver, int x, int v = 1)
    {
        modify(root[ver], root[pre_ver], 0, n, x, v);
    }

    int merge(int p, int q, int l, int r)
    {
        if (!p || !q)
            return p | q;

        int rt = ++tot;

        if (r - l == 1)
        {
            info[rt].cnt = info[p].cnt + info[q].cnt;
            return rt;
        }
        int mid = l + r >> 1;
        info[rt].l = merge(info[p].l, info[q].l, l, mid);
        info[rt].r = merge(info[p].r, info[q].r, mid, r);
        pull(rt);
        return rt;
    }
    void merge_ver(int new_ver, int u, int v)
    {
        root[new_ver] = merge(root[u], root[v], 0, n);
    }
    //将p的前k个移动给q
    void Split(int &p, int &q, int k)
    {
        if (!p)
            return;
        if (k >= info[p].cnt)
        {
            q = p;
            p = 0;
            return;
        }
        q = ++tot;
        int v = info[info[p].l].cnt;
        if (k > v)
        {
            info[q].l = info[p].l;
            info[p].l = 0;
            Split(info[p].r, info[q].r, k - v);
```

```
        }
        else if (k == v)
        {
            info[q].l = info[p].l;
            info[p].l = 0;
        }
        else
        {
            Split(info[p].l, info[q].l, k);
        }
        pull(p);
        pull(q);
    }
    void split(int new_ver, int u, int k)
    {
        root[new_ver] = 0;
        Split(root[u], root[new_ver], k);
    }
};
```

线段树套线段树

定义

二维线段树在横坐标的每个线段树结点维护一棵纵坐标线段树，适合静态、稠密矩阵的单点修改与矩形求和。以下实现采用 0 下标和左闭右开区间 `[x1, x2) × [y1, y2)`，支持初始矩阵建树、单点加、单点赋值和矩形和查询。

实现

外层结点覆盖一个横坐标区间，内层树维护该区间内各纵坐标区间的元素和。一次单点修改会更新所有经过的外层结点及其内层树；矩形查询将横坐标区间拆成若干外层结点，再在对应内层树中查询纵坐标区间。

代码

```
const int inf = 1e9;
struct Info
{
    int max;
    Info() : max(-inf) {};
    Info(int value) : max(value) {};
    Info operator+(const Info &other) const
    {
        Info res;
        res.max = std::max(max, other.max);
        return res;
    }
};
struct SegTree
{
    int n;
    vector<int> Y;
    vector<Info> info;
    SegTree() : n(0) {}
    SegTree(int _n, vector<int> y) : Y(move(y)), n(_n), info(4 * _n) {};
    int findy(int y)
    {
        return ranges::lower_bound(Y, y) - Y.begin();
    }
    void pull(int p) // 更新上移
    {
        info[p] = info[p << 1] + info[p << 1 | 1];
    }
    // 区间查询[x,y)
    Info query(int p, int l, int r, int x, int y)
    {
        if (l >= y || r <= x)
            return {};
        if (l >= x && r <= y)
        {
            return info[p];
        }
        int mid = l + r >> 1;
        return query(p << 1, l, mid, x, y) + query(p << 1 | 1, mid, r, x, y);
    };
    Info query(int x, int y)
    {
        return query(1, 0, n, x, y);
    };
    // 单点修改
    void modify(int p, int l, int r, int x, const Info &v)
    {
        if (r - l == 1)
        {
            info[p] = info[p] + v;
            return;
        }
        int mid = l + r >> 1;
        if (x < mid)
        {
            modify(p << 1, l, mid, x, v);
        }
        else
        {
            modify(p << 1 | 1, mid, r, x, v);
        }
        pull(p);
    }
    void modify(int x, int v)
    {
        modify(1, 0, n, x, Info(v));
    }

};
struct SegTree2D
{
    int n;
    vector<SegTree> info;
    SegTree2D(const vector<vector<int>> &pY)
        : n(pY.size()), info(4 * n)
    {
        build(1, 0, n, pY);
    }
    // 区间查询[x,y)
    Info query(int p, int l, int r, int xl, int xr, int yl, int yr)
    {
        if (l >= xr || r <= xl)
            return {};
        if (l >= xl && r <= xr)
        {
            int L = info[p].findy(yl);
            int R = ranges::upper_bound(info[p].Y, yr) - info[p].Y.begin();
            return info[p].query(L, R);
        }
        int mid = l + r >> 1;
        return query(p << 1, l, mid, xl, xr, yl, yr) + query(p << 1 | 1, mid, r, xl, xr, yl, yr);
```

```
    };
    Info query(int xl, int xr, int yl, int yr)
    {
        return query(1, 0, n, xl, xr, yl, yr);
    };
    // 单点修改
    void modify(int p, int l, int r, int x, int y, int v)
    {
        info[p].modify(info[p].findy(y), v);
        if (r - l == 1)
        {
            return;
        }
        int mid = l + r >> 1;
        if (x < mid)
        {
            modify(p << 1, l, mid, x, y, v);
        }
        else
        {
            modify(p << 1 | 1, mid, r, x, y, v);
        }
    }
    void modify(int x, int y, int v)
    {
        modify(1, 0, n, x, y, v);
    }
    void build(int p, int l, int r,
               const vector<vector<int>> &pY)
    {
        if (r - l == 1)
        {
            info[p] = SegTree(pY[l].size(), pY[l]);
            return;
        }

        int mid = (l + r) >> 1;
        build(p << 1, l, mid, pY);
        build(p << 1 | 1, mid, r, pY);

        vector<int> resY;
        merge(info[p << 1].Y.begin(), info[p << 1].Y.end(),
              info[p << 1 | 1].Y.begin(), info[p << 1 | 1].Y.end(),
              back_inserter(resY));

        resY.erase(unique(resY.begin(), resY.end()), resY.end());

        int sz = resY.size();
        info[p] = SegTree(sz, move(resY));
    }
};
```

```cpp
struct BIT2D
{
    int n;
    vector<vector<int>> Y, tr;

    // p：所有可能修改的点 (x, y)
    // x 必须已经离散化为 1 ~ n
    BIT2D(const vector<array<int, 2>> &p, int _n) : n(_n)
    {
        Y.resize(n + 1);
        tr.resize(n + 1);

        // 每个点预先加入它会经过的外层 BIT 节点
        for (auto [x, y] : p)
        {
            for (int i = x; i <= n; i += i & -i)
                Y[i].push_back(y);
        }

        // 每个外层节点单独离散化 y
        for (int i = 1; i <= n; i++)
        {
            ranges::sort(Y[i]);
            Y[i].erase(unique(Y[i].begin(), Y[i].end()), Y[i].end());

            tr[i].assign(Y[i].size() + 1, -inf);
        }
    }

    // 单点取 max：a[x][y] = max(a[x][y], v)
    void modify(int x, int y, int v)
    {
        for (int i = x; i <= n; i += i & -i)
        {
            int j = ranges::lower_bound(Y[i], y) - Y[i].begin() + 1;

            for (; j < tr[i].size(); j += j & -j)
                tr[i][j] = max(tr[i][j], v);
        }
    }

    // 查询所有 x' <= x 且 y' <= y 的最大值
    int query(int x, int y)
    {
        int res = -inf;

        for (int i = x; i > 0; i -= i & -i)
        {
            int j = ranges::upper_bound(Y[i], y) - Y[i].begin();

            for (; j > 0; j -= j & -j)
                res = max(res, tr[i][j]);
        }

        return res;
    }
};
```

复杂度

设矩阵大小为 $n \times m$。建树时间为 $O(nm)$；单点加、单点赋值和矩形求和均为 $O(\log n \log m)$；空间复杂度为 $O(nm)$，常数较大。

线段树的特殊应用

前缀和的最大值

```
const int inf = 4e18;
struct Info
{
    int sum, max;
    Info() : sum(), max(-inf) {};
    Info(int val, int max = -inf) : sum(val), max(max) {};
    Info operator+(const Info &other) const
    {
        Info res;
        res.sum = sum + other.sum;
        res.max = std::max(max, sum + other.max);
        return res;
    }
};
struct SegTree
{
    int n;
    vector<Info> info;
    SegTree(int _n) : n(_n), info(4 * _n) {};
    void pull(int p) // 更新上移
    {
        info[p] = info[p << 1] + info[p << 1 | 1];
    }
    // 区间查询[x,y)
    Info query(int p, int l, int r, int x, int y)
    {
        if (l >= y || r <= x)
            return {};
        if (l >= x && r <= y)
        {
            return info[p];
        }
        int mid = l + r >> 1;
        return query(p << 1, l, mid, x, y) + query(p << 1 | 1, mid, r, x, y);
    };
    Info query(int x, int y)
    {
        return query(1, 0, n, x, y);
    };
    // 单点修改
    void modify(int p, int l, int r, int x, const Info &v)
    {
        if (r - l == 1)
        {
            info[p] = v;
            return;
        }
        int mid = l + r >> 1;
        if (x < mid)
        {
            modify(p << 1, l, mid, x, v);
        }
        else
        {
            modify(p << 1 | 1, mid, r, x, v);
        }
        pull(p);
    }
    void modify(int x, Info v)
    {
        modify(1, 0, n, x, v);
    }
};
```

前缀最大值的和

```
const int inf = 1e18;
struct Info
{
    int sum, max;
    Info() : sum(), max(-inf) {};
    Info(int value) : sum(value), max(value) {};
};
struct SegTree
{
    int n;
    vector<Info> info;
    SegTree(int _n) : n(_n), info(4 * _n) {};
    int calc(int p, int l, int r, int val)
    {
        if (r - l == 1)
            return std::max(val, info[p].max);
        int mid = l + r >> 1;
        if (val >= info[p << 1].max)
        {
            int lres = val * (mid - l);
            return lres + calc(p << 1 | 1, mid, r, val);
        }
        else
        {
            int lres = calc(p << 1, l, mid, val);
            int rres = info[p].sum - info[p << 1].sum;
            return lres + rres;
        }
    }
    void pull(int p, int l, int r) // 更新上移
    {
        info[p].max = std::max(info[p << 1].max, info[p << 1 | 1].max);
        info[p].sum = info[p << 1].sum + calc(p << 1 | 1, l + r >> 1, r, info[p << 1].max);
    }
    // 区间查询[x,y)
    int query(int p, int l, int r, int x, int y, int &cur)
    {
        if (l >= y || r <= x)
            return {};
        if (l >= x && r <= y)
        {
            int res = calc(p, l, r, cur);
            cur = max(info[p].max, cur);
            return res;
        }
        int res = 0;
        int mid = l + r >> 1;
        if (x < mid)
            res += query(p << 1, l, mid, x, y, cur);

        if (y > mid)
            res += query(p << 1 | 1, mid, r, x, y, cur);

        return res;
    };
    int query(int x, int y)
    {
        int cur = -inf;
        return query(1, 0, n, x, y, cur);
    };
    // 单点修改
    void modify(int p, int l, int r, int x, const Info &v)
    {
        if (r - l == 1)
        {
            info[p] = v;
            return;
        }
        int mid = l + r >> 1;
        if (x < mid)
        {
            modify(p << 1, l, mid, x, v);
        }
        else
        {
            modify(p << 1 | 1, mid, r, x, v);
        }
        pull(p, l, r);
    }
    void modify(int x, int v)
    {
        modify(1, 0, n, x, Info(v));
    }
};
```

莫队

形式

假设 $n=m(m为询问次数)$，对于序列上的区间询问问题，如果从 $[l,r]$ 从 $O(1)$扩展到 $[l-1,r]$， $[l+1,r],[l,r-1],[l,r+1]$。那么 $O(n\sqrt{n})$的时间复杂度求出所有询问。

实现

离线后排序，顺序处理每个询问，暴力从上一个区间的答案转移到下一个区间答案（一步一步移动即可）.

排序方法：
对于区间 $[l,r]$ 以 l 所在的块为第一关键字， r为第二关键字排序。

代码

```
#include<bits/stdc++.h>
#define int long long
using namespace std;

const int N=1e6;
int len;
struct Q
{
    int l,r,idx,bl;
    bool operator <(const Q &q)const{
                  if (bl != q.bl) return bl < q.bl;
        return (bl&1) ? (r < q.r) : (r > q.r);
         }
};
vector<int>cnt(N+1);
int res=0;
void solve(){
    int n;
    cin>>n;
    len=max(1LL,(int)(n/sqrt(n)));
    vector<int>a(n+1);
    for(int i=1;i<=n;i++){
        cin>>a[i];
    }
    int q;
    cin>>q;
    vector<Q>qu(q);
    int p=0;
    for(auto &[l,r,id,bl]:qu){
        cin>>l>>r;
        id=p++;
        bl=l/len;
    }
    auto add=[&](int x){
        if(cnt[a[x]]==0)res++;
        cnt[a[x]]++;
    };
    auto del=[&](int x){
        if( cnt[a[x]]==1)res--;
        cnt[a[x]]--;
    };
    sort(qu.begin(),qu.end());
    int l=1,r=0;
    vector<int>ans(q);
    for(auto &[L,R,idx,_]:qu){
        while(l>L)add(--l);
        while(l<L)del(l++);
        while(r>R)del(r--);
        while(r<R)add(++r);
        ans[idx]=res;
    }
    for(auto x:ans)cout<<x<<'\n';
}
signed main(){
    ios::sync_with_stdio(0),cin.tie(0),cout.tie(0);
    int t=1;
    //cin>>t;
    while(t--)solve();
    return 0;
}
```

int128类定义

代码

```
using i128 = __int128;

std::ostream &operator<<(std::ostream &os, i128 n)
{
    if (n == 0)
    {
        return os << 0;
    }
    bool sign = 0;
    if (n < 0)
    {
        sign = 1;
        n = -n;
    }
    std::string s;
    while (n > 0)
    {
        s += char('0' + n % 10);
        n /= 10;
    }
    if (sign)
        s = s + '-';
    std::reverse(s.begin(), s.end());
    return os << s;
}
std::istream &operator>>(std::istream &is, i128 &n)
{
    string s;
    if (is >> s)
    {
        n = 0;
        bool sign = s[0] == '-';
        for (int i = sign; i < s.size(); i++)
        {
            n = n * 10 + (s[i] - '0');
        }
        if (sign)
            n = -n;
    }
    return is;
}
i128 toi128(const std::string &s)
{
    i128 n = 0;
    bool sign = s[0] == '-';
    for (auto c : s)
    {
        if (c == '-')
            continue;
        n = n * 10 + (c - '0');
    }
    return sign ? -n : n;
}

i128 sqrti128(i128 n)
{
    i128 l = 0, r = 2e18;
    while (l + 1 != r)
    {
        i128 mid = l + (r - l + 1) / 2;
        if (mid <= n / mid)
            l = mid;
        else
            r = mid;
    }
    return l;
}

i128 gcd(i128 a, i128 b)
{
    if (a < 0)
        a = -a;
    if (b < 0)
        b = -b;
    while (b)
    {
        a %= b;
        std::swap(a, b);
    }
    return a;
}
```

中国剩余定理（CRT）

定义

给定同余方程组 $x \equiv r_i \pmod{m_i}$，当所有模数 m_i 为正且两两互质时，方程组在模 $M = \prod m_i$ 意义下有唯一解。模板返回区间 $[0, M)$ 内的最小非负解，要求 M 能放入 `long long`。

实现

令 $M_i = M/m_i$，用扩展欧几里得算法求出 M_i 在模 m_i 下的逆元 t_i。答案为

$$x \equiv \sum_{i=0}^{n-1} r_i M_i t_i \pmod{M}.$$

输入数组使用 `0` 下标，`r` 与 `m` 长度必须相同，余数允许为负；空同余方程组返回 `0`。

代码

```
long long exgcd(long long a, long long b, long long &x, long long &y)
{
    if (!b)
    {
        x = 1, y = 0;
        return a;
    }
    long long x1, y1;
    long long g = exgcd(b, a % b, x1, y1);
    x = y1;
    y = x1 - a / b * y1;
    return g; // ax + by = gcd(a, b)
}
bool merge_crt(long long &a1, long long &m1, long long a2, long long m2)
{
    long long x, y;
    long long g = exgcd(m1, m2, x, y);
    long long c = a2 - a1;
    if (c % g)
        return false; // 差值不能被 gcd 整除，无解

    long long mod = m2 / g;
    x = (__int128)x * (c / g) % mod;
    a1 += x * m1;
    m1 *= mod; // 新模数为 lcm(m1, m2)
    a1 = (a1 % m1 + m1) % m1;
    return true;
}

// equations: {余数, 模数}
array<int,2> excrt(const vector<array<int,2>> &equations)
{
    long long a = equations[0][0];
    long long m = equations[0][1];
    for (int i = 1; i < (int)equations.size(); i++)
    {
        if (!merge_crt(a, m, equations[i][0], equations[i][1]))
            return {-1, -1};
    }
    return {a, m}; // x = a (mod m)
}
```

设方程数量为 n，时间复杂度为 $O(n \log M)$，递归栈空间复杂度为 $O(\log M)$。`__int128` 只保护中间乘法，调用者仍需保证 M 不会溢出 `long long`；模数不互质时应改用扩展中国剩余定理。

二项式反演

定义

二项式反演本质上是一种**容斥原理**的代数形式。它主要用于解决“**恰好有 k 个**”难以直接求解，但“**至多/至少有 k 个**”容易求解的问题。

- $g(k)$：通常表示“恰好”满足 k 个限制条件的方案数（直接求很难）。
- $f(k)$：通常表示“至少”或“从 n 个中选 k 个，剩下随意”的方案数（直接求容易）。

常见形式如下：

1. $f_n = \sum_{i=0}^{n} \binom{n}{i} g_i \iff g_n = \sum_{i=0}^{n} (-1)^{n-i} \binom{n}{i} f_i$
2. $f_k = \sum_{i=k}^{n} \binom{i}{k} g_i \iff g_k = \sum_{i=k}^{n} (-1)^{i-k} \binom{i}{k} f_i$ (常用：f_k 表示“至少 k 个”，g_k 表示“恰好 k 个”)

实现

代码

```cpp
#include <vector>

using i64 = long long;

template<class T>
struct Binom {
    int n;
    std::vector<T> fac, inv;
    Binom(int n) : n(n), fac(n + 1), inv(n + 1) {
        fac[0] = 1;
        for (int i = 1; i <= n; i++) fac[i] = fac[i - 1] * i;
        inv[n] = fac[n].inv(); // 假设 T 是模数类 mint，需实现 inv()
        for (int i = n; i >= 1; i--) inv[i - 1] = inv[i] * i;
    }
    T C(int n, int m) {
        if (m < 0 || m > n) return 0;
        return fac[n] * inv[m] * inv[n - m];
    }
    // 形式 2：由至少 f 数组求恰好 g 数组
    std::vector<T> solve(const std::vector<T>& f) {
        int m = f.size() - 1;
        std::vector<T> g(m + 1);
        for (int k = 0; k <= m; k++) {
            for (int i = k; i <= m; i++) {
                T res = C(i, k) * f[i];
                if ((i - k) & 1) g[k] -= res;
                else g[k] += res;
            }
        }
        return g;
    }
};
```

快速傅里叶变换 (FFT)

介绍

快速傅里叶变换 (FFT) 是一种高效计算离散傅里叶变换 (DFT) 及其逆变换 (IDFT) 的算法。它利用单位复数根的对称性与周期性，通过分治策略将多项式从**系数表示法**转换为**点值表示法**，从而将原本 $O(N^2)$ 的多项式乘法复杂度降低到 $O(N \log N)$。

功能

- **多项式乘法**：高效计算两个高次多项式的乘积（线性卷积）。
- **大整数乘法**：将高精度大整数视为多项式进行快速相乘。
- **字符串匹配**：通过构造多项式，处理带有通配符（模糊匹配）的字符串模式匹配问题。

迭代法实现

代码

```
const double pi = acos(-1.0);
using Complex = complex<double>;
struct FFT
{
    vector<int> a, b;
    FFT(vector<int> &a, vector<int> &b) : a(a), b(b) {};
    vector<int> mul()
    {
        int n = 1;
        int tot = a.size() + b.size() - 1;
        while (n < tot)
        {
            n <<= 1;
        }
        vector<Complex> fa(n), fb(n);
        for (int i = 0; i < a.size(); i++)
        {
            fa[i] = a[i];
        }
        for (int i = 0; i < b.size(); i++)
        {
            fb[i] = b[i];
        }
        dft(fa);
        dft(fb);
        for (int i = 0; i < n; i++)
        {
            fa[i] *= fb[i];
        }
        idft(fa);
        vector<int> c(tot);
        for (int i = 0; i < c.size(); i++)
        {
            c[i] = round(fa[i].real());
        }
        return c;
    }
    void dft(vector<Complex> &a)
    {
        int n = a.size();
        vector<int> rev(n);
        for (int i = 0; i < n; i++)
        {
            rev[i] = (rev[i >> 1] >> 1) | ((i & 1) ? (n >> 1) : 0);
        }
        for (int i = 0; i < n; i++)
        {
            if (i < rev[i])
                swap(a[i], a[rev[i]]);
        }
        for (int k = 1; k < n; k <<= 1)
        {
            Complex wn(cos(pi / k), sin(pi / k));
            for (int i = 0; i < n; i += 2 * k)
            {
                Complex w(1, 0);
                for (int j = 0; j < k; j++)
                {
                    Complex u = a[i + j], v = a[i + j + k] * w;
                    a[i + j] = u + v;
                    a[i + j + k] = u - v;
                    w *= wn;
                }
            }
        }
    }
    void idft(vector<Complex> &a)
    {
        int n = a.size();
        reverse(a.begin() + 1, a.end());
        dft(a);
        for (int i = 0; i < n; i++)
        {
            a[i] /= n;
        }
    }
};
```

快速数论变换

定义

NTT (Number Theoretic Transform, 快速数论变换) 是在数论值域（通常基于大质数模数）下的快速傅里叶变换（FFT）。它利用**原根**替代了复数域中的单位根，从而完美解决了 FFT 带来的浮点数精度误差问题。

实现

以下为模数 $P = 998244353$（原根 $G = 3$）下的通用 NTT 结构体封装实现。

代码

```
namespace NTT
{
    const int mod = 998244353, g = 3;
    int exp(int a, int b)
    {
        int ans = 1;
        while (b)
        {
            if (b & 1)
                ans = a * ans % mod;
            a = a * a % mod;
            b >>= 1;
        }
        return ans;
    }
    void dft(vector<int> &a)
    {
        int n = a.size();
        vector<int> rev(n);
        for (int i = 0; i < n; i++)
        {
            rev[i] = (rev[i >> 1] >> 1) | ((i & 1) ? (n >> 1) : 0);
        }
        for (int i = 0; i < n; i++)
        {
            if (i < rev[i])
                swap(a[i], a[rev[i]]);
        }
        for (int k = 1; k < n; k <<= 1)
        {
            int wn = exp(g, (mod - 1) / (2 * k));
            for (int i = 0; i < n; i += 2 * k)
            {
                int w = 1;
                for (int j = 0; j < k; j++)
                {
                    int u = a[i + j], v = a[i + j + k] * w % mod;
                    a[i + j] = (u + v) % mod;
                    a[i + j + k] = (u - v + mod) % mod;
                    (w *= wn) %= mod;
                }
            }
        }
    }
    void idft(vector<int> &a)
    {
        int n = a.size();
        reverse(a.begin() + 1, a.end());
        dft(a);
        int inv = exp(n, mod - 2);
        for (int i = 0; i < n; i++)
        {
            (a[i] *= inv) %= mod;
        }
    }
    vector<int> mul(vector<int> a, vector<int> b)
    {
        int tot = a.size() + b.size() - 1;
        int n = 1;
        while (n < tot)
        {
            n <<= 1;
        }
        a.resize(n), b.resize(n);
        dft(a), dft(b);
        for (int i = 0; i < n; i++)
        {
            (a[i] *= b[i]) %= mod;
        }
        idft(a);
        a.resize(tot);
        return a;
    }
};
```

扩展欧几里得算法（exgcd）

定义

扩展欧几里得算法用于求整数 a, b 的最大公约数 g，并构造一组整数 x, y，使得 $ax + by = g$。模板要求 a, b 非负且不同时为 0，时间复杂度为 $O(\log \min(a, b))$。

实现

递归求出 $bx_1 + (a \bmod b)y_1 = g$。由 $a \bmod b = a - \lfloor a/b \rfloor b$ 可得 $x = y_1$、$y = x_1 - \lfloor a/b \rfloor y_1$；当 $b = 0$ 时取 $x = 1, y = 0$。

代码

```
long long exgcd(long long a, long long b, long long &x, long long &y) {
    if (b == 0) {
        x = 1;
        y = 0;
        return a;
    }
    long long x1, y1;
    long long g = exgcd(b, a % b, x1, y1);
    x = y1;
    y = x1 - a / b * y1;
    return g;
}
```

递归栈空间复杂度为 $O(\log \min(a, b))$。若输入或中间乘积可能超出 `long long`，需要将系数和运算过程改为 `__int128`。

拉格朗日插值

定义

已知 n 个横坐标两两不同的点 (x_i, y_i)，可唯一确定一个次数小于 n 的多项式。下面在质数模数 `MOD` 下计算该多项式在 `k` 处的值；要求所有 `x_i` 模 `MOD` 后互不相同。

时间复杂度为 $O(n^2 + n \log MOD)$，其中 n 为插值点个数；空间复杂度为 $O(n)$。

实现

根据

$$f(k) = \sum_{i=0}^{n-1} y_i \prod_{j \neq i} \frac{k - x_j}{x_i - x_j}$$

逐项计算分子和分母。`x`、`y` 均为 0 下标数组；输入值可以是任意整数，函数会将其规范到模 `MOD` 的范围内。

代码

```cpp
#include <cassert>
#include <vector>

using i64 = long long;

constexpr i64 MOD = 998244353;

i64 mod_pow(i64 a, i64 e) {
    i64 res = 1;
    while (e > 0) {
        if (e & 1) res = res * a % MOD;
        a = a * a % MOD;
        e >>= 1;
    }
    return res;
}

i64 norm(i64 x) {
    x %= MOD;
    return x < 0 ? x + MOD : x;
}

// 返回经过 (x[i], y[i]) 的多项式在 k 处的值。
i64 lagrange(const std::vector<i64>& x, const std::vector<i64>& y, i64 k) {
    int n = x.size();
    assert(n > 0 && y.size() == x.size());

    std::vector<i64> xs(n), ys(n);
    for (int i = 0; i < n; i++) {
        xs[i] = norm(x[i]);
        ys[i] = norm(y[i]);
    }
    k = norm(k);

    for (int i = 0; i < n; i++) {
        if (xs[i] == k) return ys[i];
    }

    i64 ans = 0;
    for (int i = 0; i < n; i++) {
        i64 num = 1, den = 1;
        for (int j = 0; j < n; j++) {
            if (i == j) continue;
            num = num * norm(k - xs[j]) % MOD;
            den = den * norm(xs[i] - xs[j]) % MOD;
        }
        assert(den != 0);  // 保证所有横坐标模 MOD 后互不相同。
        ans = (ans + ys[i] * num % MOD * mod_pow(den, MOD - 2)) % MOD;
    }
    return ans;
}
```

指数生成函数 (EGF) 与有标号计数

定义

指数生成函数（Exponential Generating Function）主要用于解决**有标号计数**问题。将序列 $a_0, a_1, a_2, \dots$ 映射为形式幂级数

$$A(x) = \sum_{n=0}^{\infty} a_n \frac{x^n}{n!},$$

其中 a_n 表示在 n 个不同标号上构造目标结构的方案数。

核心算子：有标号乘积

若 $C(x) = A(x)B(x)$，则先从 n 个标号中选择 k 个交给第一类结构，再用剩余标号构造第二类结构，因此

$$c_n = \sum_{k=0}^{n} \binom{n}{k} a_k b_{n-k}.$$

若 $A(0) = 0$，由若干个 $\mathcal{A}$ 对象组成的无序集合对应

$$\mathrm{SET}(\mathcal{A}) \longleftrightarrow \exp(A(x)).$$

常用的标号序列与标号环构造分别对应 $1/(1 - A(x))$ 与 $\log(1/(1 - A(x)))$。

实现

将 EGF 系数 a_i 除以 $i!$ 后即可使用普通多项式卷积，最后再将第 i 项乘以 $i!$。下面给出 $O(NM)$ 的通用实现；数组使用 `0` 下标，`a[i]` 存储计数 a_i 而非 $a_i/i!$。`T` 需要支持四则运算，且最高次数的阶乘必须可逆；在质数模数下通常要求最高次数小于模数。

代码

```
#include <cassert>
#include <vector>

template<class T>
struct ExponentialGeneratingFunction {
    int n;
    std::vector<T> fac, ifac;

    explicit ExponentialGeneratingFunction(int n)
        : n(n), fac(n + 1), ifac(n + 1) {
        fac[0] = T(1);
        for (int i = 1; i <= n; i++) {
            fac[i] = fac[i - 1] * T(i);
        }
        ifac[n] = T(1) / fac[n];
        for (int i = n; i >= 1; i--) {
            ifac[i - 1] = ifac[i] * T(i);
        }
    }

    std::vector<T> multiply(const std::vector<T>& a,
                            const std::vector<T>& b) const {
        if (a.empty() || b.empty()) {
            return {};
        }
        int degree = static_cast<int>(a.size() + b.size()) - 2;
        assert(degree <= n);

        std::vector<T> pa(a.size()), pb(b.size());
        for (int i = 0; i < static_cast<int>(a.size()); i++) {
            pa[i] = a[i] * ifac[i];
        }
        for (int i = 0; i < static_cast<int>(b.size()); i++) {
            pb[i] = b[i] * ifac[i];
        }

        std::vector<T> c(degree + 1);
        for (int i = 0; i < static_cast<int>(a.size()); i++) {
            for (int j = 0; j < static_cast<int>(b.size()); j++) {
                c[i + j] += pa[i] * pb[j];
            }
        }
        for (int i = 0; i <= degree; i++) {
            c[i] *= fac[i];
        }
        return c;
    }
};
```

若两个序列的最高次数分别为 N 和 M，时间复杂度为 $O(NM)$，空间复杂度为 $O(N+M)$；将代码中的普通卷积替换为 NTT 后可降至 $O((N+M)\log(N+M))$。

数论分块（整除分块）

定义

数论分块用于枚举 $\lfloor n/i \rfloor$ 的所有连续等值区间。对当前左端点 l，令 $q=\lfloor n/l \rfloor$，则相同商对应的最大右端点为

$$r=\left\lfloor \frac{n}{q} \right\rfloor .$$

$\lfloor n/i \rfloor$ 只有 $O(\sqrt{n})$ 种取值，因此可以在 $O(\sqrt{n})$ 次分块内完成形如

$$\sum_{i=1}^{n} f\left(\left\lfloor \frac{n}{i} \right\rfloor\right) g(i)$$

的求和。若 $G(i)=\sum_{j=1}^{i} g(j)$，区间 $[l,r]$ 的贡献为 $f(q)\big(G(r)-G(l-1)\big)$。

实现

单变量版本将每个分块的 `l`、`r`、`q` 传给回调函数。双变量版本枚举 $1 \le i \le \min(n,m)$，公共右端点取

$$r=\min\left(\left\lfloor \frac{n}{\lfloor n/l \rfloor} \right\rfloor, \left\lfloor \frac{m}{\lfloor m/l \rfloor} \right\rfloor\right),$$

并向回调函数传入 `l`、`r`、`n / l`、`m / l`。所有区间均为 `1` 下标闭区间；要求参数非负，参数为 0 时不产生分块。回调中的乘法与累加可能溢出，调用者需要按答案范围选择 `long long` 或 `__int128`。

单变量版本的时间复杂度为 $O(\sqrt{n})$，双变量版本的时间复杂度为 $O(\sqrt{n}+\sqrt{m})$；除回调函数自身使用的空间外，额外空间复杂度为 $O(1)$。

普通生成函数 (OGF) 与序列构造

定义

普通生成函数（Ordinary Generating Functions）主要用于解决**无标号计数**问题。将序列 $a_0, a_1, a_2, \dots$ 映射为一个形式幂级数：

$$A(x) = \sum_{n=0}^{\infty} a_n x^n$$

其中 x^n 的指数代表物品的“大小”或“长度”，系数 a_n 代表方案数。

核心算子：序列构造 $SEQ(\mathcal{A})$

若单个物品的生成函数为 $A(x)$，且满足 $[x^0]A(x) = 0$（即没有大小为 0 的物品），则由该物品排成一列构成的序列生成函数为：

$$G(x) = 1 + A(x) + A^2(x) + A^3(x) + \dots = \frac{1}{1 - A(x)}$$

实现

在多项式工业题中，通常使用**多项式求逆**实现；在组合数学推导中，通常利用闭式进行系数提取。

代码 (系数提取逻辑)

```
// 场景：已知每种物品的选法 A(x)，求总长度为 n 的排列方案数
// 对应公式：G(x) = 1 / (1 - A(x))
// 如果 A(x) 是简单多项式，通常使用线性递推或 Bostan-Mori 提取系数
// 如果 A(x) 项数较多，使用多项式求逆 O(N log N)
vector<int> PolyInv(vector<int> A, int n); // 标准模板
```

广义二项式定理 (Generalized Binomial Theorem)

定义

将二项式定理 $(1+x)^n$ 推广到 n 为任意实数（负数、分数）的情况：

$$(1+x)^\alpha = \sum_{k=0}^{\infty} \binom{\alpha}{k} x^k$$

其中广义组合数的展开为：

$$\binom{\alpha}{k} = \frac{\alpha(\alpha-1)(\alpha-2)\dots(\alpha-k+1)}{k!}$$

XCPC 最常用变形（负二项式展开）：

对于正整数 n，处理分母上的 $(1-x)^n$ 时：

$$[x^k]\frac{1}{(1-x)^n} = \binom{n+k-1}{k} = \binom{n+k-1}{n-1}$$

这个公式常用于将“隔板法”问题转化为生成函数系数提取。

实现

通过预处理阶乘和逆元，在 $O(1)$ 或 $O(k)$ 时间内提取特定项。

代码 (广义组合数计算)

```
// 计算广义组合数 C(alpha, k)，alpha 可以为负
int generalized_C(int alpha, int k) {
    if (k < 0) return 0;
    if (alpha >= 0 && alpha < k) return 0;
    if (alpha >= 0) return C(alpha, k); // 普通组合数

    // 若 alpha 为负数，设 n = -alpha
    // 使用公式：C(-n, k) = (-1)^k * C(n+k-1, k)
    int n = -alpha;
    int res = C(n + k - 1, k);
    if (k & 1) return (mod - res) % mod;
    return res;
}

// 提取 [x^L] (1-x)^{-n} 的系数
int extract_negative_pow(int n, int L) {
    return C(n + L - 1, n - 1);
}
```

狄利克雷卷积

定义

两个算术函数 f, g 的狄利克雷卷积定义为

$$(f * g)(n) = \sum_{d|n} f(d) g\left(\frac{n}{d}\right).$$

狄利克雷卷积满足交换律、结合律和分配律，单位元为 $\varepsilon(n) = [n = 1]$。若 $f(1)$ 在当前数域中可逆，则 f 存在唯一卷积逆元；两个积性函数的卷积仍是积性函数。

常用恒等式为

$$\mathbf{1} * \mu = \varepsilon, \qquad \mathbf{1} * \varphi = \mathrm{id},$$

其中 $\mathbf{1}(n) = 1$，$\mathrm{id}(n) = n$。

实现

数组使用 `1` 下标，`f[0]`、`g[0]` 无意义。枚举因子 d 和另一因子 k，将贡献累加到 dk；求逆时按下标递增维护已知项对后续倍数的贡献。`T` 需要支持四则运算，调用 `dirichlet_inverse` 时还要求 `f[1]` 可逆。

代码

```
#include <cassert>
#include <vector>

template<class T>
std::vector<T> dirichlet_convolution(const std::vector<T>& f,
                                     const std::vector<T>& g,
                                     int n) {
    assert(static_cast<int>(f.size()) > n);
    assert(static_cast<int>(g.size()) > n);
    std::vector<T> h(n + 1);
    for (int d = 1; d <= n; d++) {
        for (int k = 1; d * k <= n; k++) {
            h[d * k] += f[d] * g[k];
        }
    }
    return h;
}

template<class T>
std::vector<T> dirichlet_inverse(const std::vector<T>& f) {
    int n = static_cast<int>(f.size()) - 1;
    assert(n >= 1);

    std::vector<T> g(n + 1), sum(n + 1);
    T inv_f1 = T(1) / f[1];
    g[1] = inv_f1;

    for (int k = 1; k <= n; k++) {
        if (k > 1) {
            g[k] = -inv_f1 * sum[k];
        }
        for (int d = 2; d * k <= n; d++) {
            sum[d * k] += f[d] * g[k];
        }
    }
    return g;
}
```

计算 $1 \sim N$ 的卷积或卷积逆均需 $O(N \log N)$ 时间和 $O(N)$ 空间。

狄利克雷生成函数 (DGF) 与算术函数

定义

狄利克雷生成函数（Dirichlet Generating Function）主要用于研究定义在正整数上的算术函数。将序列 $a_1, a_2, a_3, \ldots$ 映射为形式级数

$$A(s) = \sum_{n=1}^{\infty} \frac{a_n}{n^s}.$$

在组合恒等式中通常只关心形式系数；若把它视为复函数，则还需单独判断级数的收敛域。

核心算子：狄利克雷卷积

若 $A(s)$ 与 $B(s)$ 的系数分别为 a_n 与 b_n，则

$$A(s)B(s) = \sum_{n=1}^{\infty} \frac{1}{n^s} \sum_{d \mid n} a_d b_{n/d}.$$

因此，狄利克雷生成函数的乘法对应系数序列的狄利克雷卷积。若 a 是积性函数，还可以按质数分解为欧拉乘积

$$A(s) = \prod_{p} \left(\sum_{k=0}^{\infty} \frac{a_{p^k}}{p^{ks}} \right).$$

常用对应关系为

$$\mathbf{1} \longleftrightarrow \zeta(s), \qquad \varepsilon \longleftrightarrow 1, \qquad \mu \longleftrightarrow \frac{1}{\zeta(s)}, \qquad \mathrm{id}_k \longleftrightarrow \zeta(s-k),$$

其中 $\mathbf{1}(n) = 1$，$\varepsilon(n) = [n = 1]$，$\mathrm{id}_k(n) = n^k$。

实现

竞赛中通常不直接计算无穷级数，而是维护 $1 \sim N$ 的系数。下面的组件将两个截断狄利克雷级数相乘，数组使用 `1` 下标， `coef[0]` 无意义； `T` 需要支持默认构造、加法与乘法。

代码

```cpp
#include <vector>

template<class T>
struct DirichletSeries {
    int n;
    std::vector<T> coef;

    explicit DirichletSeries(int n = 0) : n(n), coef(n + 1) {}

    T& operator[](int i) {
        return coef[i];
    }

    const T& operator[](int i) const {
        return coef[i];
    }

    static DirichletSeries multiply(const DirichletSeries& a,
                                    const DirichletSeries& b,
                                    int limit) {
        DirichletSeries c(limit);
        for (int d = 1; d <= limit && d <= a.n; d++) {
            for (int k = 1; d * k <= limit && k <= b.n; k++) {
                c[d * k] += a[d] * b[k];
            }
        }
        return c;
    }
};
```

计算前 N 项乘积的时间复杂度为 $O(N \log N)$，空间复杂度为 $O(N)$。

矩阵快速幂

结构体实现

使用 `std::vector` 动态维护矩阵，通过结构体封装重载乘法运算符，代码简洁且通用性强。

代码

```
const int mod = 1e9 + 7;
struct Mat
{
    int n;
    vector<vector<int>> M;
    Mat(int r) : n(r), M(r, vector<int>(r)) {};
    Mat unit()
    {
        Mat res(n);
        for (int i = 0; i < n; i++)
            res.M[i][i] = 1;
        return res;
    }
    Mat operator*(const Mat &b) const
    {
        Mat res(n);
        for (int i = 0; i < n; i++)
        {
            for (int j = 0; j < n; j++)
            {
                for (int k = 0; k < n; k++)
                {
                    res.M[i][j] += M[i][k] * b.M[k][j] % mod;
                    res.M[i][j] %= mod;
                }
            }
        }
        return res;
    }
    void print()
    {
        for (int i = 0; i < n; i++)
        {
            for (int j = 0; j < n; j++)
            {
                cout << M[i][j] << " \n"[j == n - 1];
            }
        }
    }
};
Mat exp(Mat a, int b)
{
    Mat ans(a.n);
    ans = ans.unit();
    while (b)
    {
        if (b & 1)
            ans = ans * a;
        a = a * a;
        b >>= 1;
    }
    return ans;
}
```

线性基

定义

称线性空间 V 的一个极大线性无关组为 V 的一组 **Hamel 基** 或 **线性基**，简称 **基**。

实现1（贪心法）

对原集合的每个数 p
转为二进制，从高位向低位扫，对于第 x
位是 1
的，如果 a_x 不存在，那么令 $a_x \leftarrow p$ 并结束扫描，如果存在，令$p \leftarrow p \operatorname{xor} a_x$
.

查询原集合内任意几个元素 xor 的最大值，只需将线性基从高位向低位扫，若 xor 上当前扫到的 a_x 答案变大，就把答案异或上 a_x。

代码

```
struct Basis
{
    long long p[64], d[64];
    int cnt = 0;
    bool can_zero;
    Basis()
    {
        memset(p, 0, sizeof(p));
        can_zero = 0;
    }
    // 插入元素
    void insert(long long x)
    {
        for (int i = 62; i >= 0; i--)
        {
            if (!(x >> i))
                continue;
            if (!p[i])
            {
                p[i] = x;
                return;
            }
            x ^= p[i];
        }
        can_zero = 1;
    }
    // 查询最大异或值
    long long query()
    {
        long long res = 0;
        for (int i = 62; i >= 0; i--)
        {
            res = max(res, res ^ p[i]);
        }
        return res;
    }
    // 合并线性基
    void merge(const Basis &b)
    {
        for (int i = 62; i >= 0; i--)
        {
            if (b.p[i])
                insert(b.p[i]);
        }
    }
    void rebuild()
    {
        cnt = 0;
        for (int i = 62; i >= 0; i--)
        {
            for (int j = i - 1; j >= 0; j--)
            {
                if ((p[i] >> j) & 1)
                    p[i] ^= p[j];
            }
        }

        for (int i = 0; i <= 62; i++)
        {
            if (p[i])
                d[cnt++] = p[i];
        }
    }

    long long kth(long long k)
    {
        if (can_zero)
            k--; // 如果原集合能异或出0，第k小要减1
        if (k >= (1LL << cnt))
            return -1;
        long long res = 0;
        for (int i = 0; i < cnt; i++)
        {
            if ((k >> i) & 1)
                res ^= d[i];
        }
        return res;
    }
};
```

实现2（高斯消元）

代码

```
#include <bits/stdc++.h>
#define int long long
using namespace std;

void solve()
{
    int n;
    cin >> n;
    vector<int> a(n);
    for (auto &x : a)
        cin >> x;
    int row = 0;
    auto gauss = [&]()
    {
        for (int i = 60; i >= 0; i--)
        {
            for (int j = row; j < n; j++)
            {
                if ((a[j] >> i) & 1)
                {
                    swap(a[j], a[row]);
                    break;
                }
            }
            if (!((a[row] >> i) & 1))
                continue;
            for (int j = 0; j < n; j++)
            {
                if (j != row && (a[j] >> i) & 1)
                    a[j] ^= a[row];
            }
            row++;
            if (row == n)
                break;
        }
    };
    gauss();
    int ans = 0;
    for (int i = 0; i < row; i++)
        ans ^= a[i];
    cout << ans << '\n';
}
signed main()
{
    ios::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    int t = 1;
    while (t--)
        solve();
    return 0;
}
```

线性筛（欧拉筛）

定义

线性筛按每个合数的最小质因子唯一生成该合数，可在 $O(N)$ 时间内筛出 $1 \sim N$ 的全部质数。利用相同的转移还能同时预处理最小质因子、欧拉函数 φ 和莫比乌斯函数 μ。

实现

对每个 i 只枚举满足 $p \le \operatorname{minp}(i)$ 的质数 p。此时 p 是 ip 的最小质因子，因此每个合数只会被筛到一次：

- 若 $p = \operatorname{minp}(i)$，则 $p \mid i$，有 $\varphi(ip) = p\varphi(i)$、$\mu(ip) = 0$。
- 若 $p < \operatorname{minp}(i)$，则 $\gcd(i, p) = 1$，有 $\varphi(ip) = (p-1)\varphi(i)$、$\mu(ip) = -\mu(i)$。

数组使用 `1` 下标，`minp[0]`、`phi[0]` 和 `mu[0]` 无意义。`factorize(x)` 要求 $1 \le x \le N$；`x=1` 时返回空分解。

代码

```
const int N = 1e7;
vector<int> vis(N + 1), prime;
void init()
{
    for (int i = 2; i <= N; i++)
    {
        if (!vis[i])
        {
            prime.push_back(i);
        }
        for (auto p : prime)
        {
            if (i * p > N)
                break;
            vis[i * p] = 1;
            if (i % p == 0)
                    break;
        }
    }
}
```

预处理时间复杂度为 $O(N)$，空间复杂度为 $O(N)$；单次分解 x 的时间复杂度为 $O(\log x)$，返回结果按质因子从小到大排列。

莫比乌斯反演

定义

莫比乌斯反演用于从整除偏序上的前缀和恢复原函数。莫比乌斯函数满足 $\sum_{d|n}\mu(d)=[n=1]$，其中

$$\mu(n)=\begin{cases}1, & n=1,\\ 0, & n\text{ 含有平方因子},\\ (-1)^k, & n\text{ 是 }k\text{ 个不同质数的乘积}.\end{cases}$$

常用的两种反演形式为

$$F(n)=\sum_{d|n}f(d)\iff f(n)=\sum_{d|n}\mu(d)F\left(\frac{n}{d}\right),$$

以及

$$F(n)=\sum_{n|d}f(d)\iff f(n)=\sum_{n|d}\mu\left(\frac{d}{n}\right)F(d).$$

常见数论函数的反演形式

记 $\varepsilon(n)=[n=1]$，φ 为欧拉函数，$\sigma_k(n)=\sum_{d|n}d^k$。常用恒等式如下：

- 莫比乌斯函数是常函数 1 在狄利克雷卷积下的逆元：

$$\sum_{d|n}\mu(d)=\varepsilon(n).$$

- 欧拉函数：

$$\sum_{d|n}\varphi(d)=n\iff\varphi(n)=\sum_{d|n}\mu(d)\frac{n}{d}=n\sum_{d|n}\frac{\mu(d)}{d}.$$

- 约数幂和函数，其中 k 为非负整数：

$$\sigma_k(n)=\sum_{d|n}d^k\iff n^k=\sum_{d|n}\mu(d)\sigma_k\left(\frac{n}{d}\right).$$

- 互质指示函数：

$$[\gcd(a,b)=1]=\sum_{d|\gcd(a,b)}\mu(d).$$

实现

线性筛维护每个数的最小质因子转移：若质数 $p\mid i$，则 $p^2\mid ip$，令 $\mu(ip)=0$ 并停止枚举；否则令 $\mu(ip)=-\mu(i)$。返回数组使用 `1` 下标，要求 $N\geq 0$，其中 `mu[0]` 无意义。

代码

```
const int N = 1e7;
vector<int> vis(N + 1), prime, mu(N + 1);
void init()
{
    mu[1] = 1;
    for (int i = 2; i <= N; i++)
    {
        if (!vis[i])
        {
            prime.push_back(i);
            mu[i] = -1;
        }
        for (auto p : prime)
        {
            if (i * p > N)
                break;
            vis[i * p] = 1;
            if (i % p == 0)
            {
                mu[i * p] = 0;
                break;
            }
            mu[i * p] = mu[i] * mu[p];
        }
    }
}
```

预处理 $1 \sim N$ 的莫比乌斯函数时间复杂度为 $O(N)$，空间复杂度为 $O(N)$。

字符串哈希 (String Hash)

定义

将一个字符串映射为一个整数，用于快速比较字符串是否相等。通过预处理前缀哈希值，可以在 $O(1)$ 时间内获取任意子串的哈希值。

时间复杂度:

- 预处理：$O(n)$
- 查询子串：$O(1)$

实现 (单模数自然溢出/大模数)

代码

```
#include <bits/stdc++.h>
using namespace std;

using u64 = unsigned long long;

const u64 base = chrono::steady_clock::now().time_since_epoch().count() | 1;
struct Hash {
    int n;
    vector<u64> h, p;

    Hash(string s) : n(s.size()), h(n + 1), p(n + 1) {
        p[0] = 1;
        for (int i = 0; i < n; i++) {
            p[i + 1] = p[i] * bs;
            h[i + 1] = h[i] * bs + s[i];
        }
    }

    u64 get(int l, int r) { // [l, r) 左闭右开
        return h[r] - h[l] * p[r - l];
    }
};
```

```cpp
#include <bits/stdc++.h>

using u64 = unsigned long long;
using u128 = __uint128_t;

constexpr u64 MOD = (1ULL << 61) - 1;

u64 base = std::chrono::steady_clock::now().time_since_epoch().count() % (MOD / 2) + MOD / 4 | 1;

struct Hash {
    std::vector<u64> h, p;
    Hash(const std::string &s) {
        int n = s.size();
        h.resize(n + 1);
        p.assign(n + 1, 1);
        for (int i = 0; i < n; i++) {
            h[i + 1] = ((u128)h[i] * base + s[i]) % MOD;
            p[i + 1] = (u128)p[i] * base % MOD;
        }
    }
    // 查询 [l, r) 的哈希值，下标从 0 开始
    u64 get(int l, int r) {
        u64 res = h[r] + MOD - (u128)h[l] * p[r - l] % MOD;
        return res >= MOD ? res - MOD : res;
    }
};
```

双哈希

```cpp
#include <bits/stdc++.h>

using i64 = long long;

struct Hash {
    static constexpr int M1 = 1E9 + 7, M2 = 1E9 + 9;
    static int B1, B2;
    std::vector<int> h1, p1, h2, p2;

    Hash(const std::string &s) {
        int n = s.size();
        h1.resize(n + 1); p1.resize(n + 1);
        h2.resize(n + 1); p2.resize(n + 1);
        h1[0] = h2[0] = 0;
        p1[0] = p2[0] = 1;
        for (int i = 0; i < n; i++) {
            h1[i + 1] = (1LL * h1[i] * B1 + s[i]) % M1;
            p1[i + 1] = 1LL * p1[i] * B1 % M1;
            h2[i + 1] = (1LL * h2[i] * B2 + s[i]) % M2;
            p2[i + 1] = 1LL * p2[i] * B2 % M2;
        }
    }

    // 返回 [l, r) 的双哈希值
    std::array<int, 2> get(int l, int r) {
        int res1 = (h1[r] - 1LL * h1[l] * p1[r - l] % M1 + M1) % M1;
        int res2 = (h2[r] - 1LL * h2[l] * p2[r - l] % M2 + M2) % M2;
        return {res1, res2};
    }
};

// 随机化底数初始化
int Hash::B1 = std::chrono::steady_clock::now().time_since_epoch().count() % 1000 + 131;
int Hash::B2 = std::chrono::steady_clock::now().time_since_epoch().count() % 1000 + 13331;
```

异或哈希 (Xor Hashing)

用于判断集合是否相同或处理子段出现次数偶数次等问题。通过给每个权值分配随机 64 位整数，将集合比较转化为异或和比较。

基础实现

使用 `std::mt19937_64` 生成随机权值，并用结构体封装哈希逻辑。

代码

```cpp
using u64 = unsigned long long;

std::mt19937_64 rng(std::chrono::steady_clock::now().time_since_epoch().count());

// 获取一个随机的 64 位整数
u64 get_random() {
    return rng();
}
```

扫描线 (Scanning Line)

矩形面积并

通过垂直于 x 轴的直线自左向右扫描，将并集图形划分为若干矩形。利用线段树维护纵坐标方向上的覆盖长度。

代码

```
#include <bits/stdc++.h>
#define int long long
using namespace std;

const int inf = 1e18;
struct Info
{
    int cnt, len;
    int size;
    Info() : cnt(0), len(0) {};
};
struct SegTree
{
    int n;
    vector<Info> info;
    vector<int> vy;
    SegTree(vector<int> vy) : n(vy.size()), info(4 * vy.size()), vy(vy) {};
    void pull(int p, int l, int r)
    {
        if (info[p].cnt)
            info[p].len = vy[r] - vy[l];
        else
        {
            if (r - l == 1)
                info[p].len = 0;
            else
                info[p].len = info[p << 1 | 1].len + info[p << 1].len;
        }
    }
    void modify(int p, int l, int r, int x, int y, int v)
    {
        if (vy[l] >= y || vy[r] <= x)
            return;
        if (vy[l] >= x && vy[r] <= y)
        {
            info[p].cnt += v;
            pull(p, l, r);
            return;
        }
        int mid = (l + r) >> 1;
        modify(p << 1, l, mid, x, y, v);
        modify(p << 1 | 1, mid, r, x, y, v);
        pull(p, l, r);
    }
    void modify(int x, int y, int v)
    {
        modify(1, 0, n - 1, x, y, v);
    }
    int get_len()
    {
        return info[1].len;
    }
};
void solve()
{
    int n;
    cin >> n;
    vector<array<int, 4>> a;
    vector<int> vy;
    for (int i = 1; i <= n; i++)
    {
        int x1, x2, y1, y2;
        cin >> x1 >> y1 >> x2 >> y2;
        a.push_back({x1, y1, y2, 1});
        a.push_back({x2, y1, y2, -1});
        vy.push_back(y1), vy.push_back(y2);
    }
    ranges::sort(vy);
    vy.erase(unique(vy.begin(), vy.end()), vy.end());
    ranges::sort(a, [&](const array<int, 4> &a, const array<int, 4> &b)
                 { return a[0] < b[0]; });
    SegTree T(vy);
    int ans = 0;
    for (int i = 0; i < a.size() - 1; i++)
    {
        auto [x, y1, yr, v] = a[i];
        int nxtx = a[i + 1][0];
        T.modify(y1, yr, v);
        ans += T.get_len() * (nxtx - x);
    }
    cout << ans << '\n';
}
signed main()
{
    ios::sync_with_stdio(0), cout.tie(0), cin.tie(0);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

指数哈希

利用多项式哈希将字符串映射为数值。给定字符串 S，其哈希值计算公式为：

$$H(S) = \sum_{i=0}^{n-1} S[i] \cdot P^{n-1-i} \pmod{M}$$

通过预处理前缀哈希值和 P 的幂次，可以在 $O(1)$ 时间内获取任意子串的哈希值。

实现

为了极致的安全性，通常使用两个大质数作为模数（如 10^9+7 和 10^9+9）以及随机的底数 P。

代码

```
using u64 = unsigned long long;
using u128 = __uint128_t;

namespace Hash {
    const u64 MOD = (1ULL << 61) - 1;
    // 使用高精度时钟生成随机底数，并保证是奇数 (| 1)
    const u64 B = chrono::steady_clock::now().time_since_epoch().count() | 1;

    // 安全乘法
    inline u64 mul(u64 a, u64 b) {
        return (u64)((u128)a * b % MOD);
    }

    // 安全加法 (替代 C++ 原生的 +)
    inline u64 add(u64 a, u64 b) {
        a += b;
        if (a >= MOD) a -= MOD;
        return a;
    }

    // 安全减法 (替代 C++ 原生的 -，计算相反数用 sub(0, x))
    inline u64 sub(u64 a, u64 b) {
        if (a < b) a += MOD;
        a -= b;
        return a;
    }

    // 快速幂
    inline u64 power(u64 a, u64 b) {
        u64 res = 1;
        while (b) {
            if (b & 1) res = mul(res, a);
            a = mul(a, a);
            b >>= 1;
        }
        return res;
    }

    // 求数值 x 的哈希值 B^x % MOD
    inline u64 get(u64 x) {
        return power(B, x);
    }
}
```

1. 求单点哈希值：

```
u64 h = ExpHash::get(x);
```

2. 求哈希的相反数（完美解决你之前遇到的死循环和 2^{64} 溢出）：

```
u64 neg_h = ExpHash::sub(0, ExpHash::get(x));
```

随机打乱数组

```
random_device rd;
mt19937 g(rd());
ranges::shuffle(a, g);
```

计算几何

```cpp
#include <bits/stdc++.h>
using namespace std;

const double eps = 1e-9;
const double pi = acos(-1.0);

using T = double;
int sign(T x)
{
    // x为int改成x==0
    if (fabs(x) < eps)
        return 0;
    return x < 0 ? -1 : 1;
}

int cmp(T x, T y)
{
    return sign(x - y);
}

double Acos(double x)
{
    if (x < -1)
        return pi;
    if (x > 1)
        return 0;
    return acos(x);
}

// ========================================
// 点与向量的结构体
// ========================================
struct Point;
using Vector = Point;

struct Point
{
    T x, y;
    Point(T x = 0, T y = 0) : x(x), y(y) {}

    Point operator+(const Point &b) const { return Point(x + b.x, y + b.y); }
    Point operator-(const Point &b) const { return Point(x - b.x, y - b.y); }
    Point operator*(T k) const { return Point(x * k, y * k); }
    Point operator/(T k) const { return Point(x / k, y / k); }

    friend istream &operator>>(istream &in, Point &p)
    {
        in >> p.x >> p.y;
        return in;
    }

    bool operator==(const Point &b) const
    {
        return cmp(x, b.x) == 0 && cmp(y, b.y) == 0;
    }

    // 点的大小比较：以 x 为第一关键字，y 为第二关键字（常用于排序）
    bool operator<(const Point &b) const
    {
        int c = cmp(x, b.x);
        if (c != 0)
            return c == -1;
        return cmp(y, b.y) == -1;
    }

    // 计算向量模长（两点间距离）
    double len() const
    {
        return hypot(x, y);
    }

    // 计算向量模长的平方
    T len2() const
    {
        return x * x + y * y;
    }

    // 求当前向量的法向量（逆时针旋转 90 度）
    Vector normal() const
    {
        return Point(-y, x);
    }

    // 求该点/向量关于原点的极角，范围在 [-pi, pi] 之间
    double p_angle() const
    {
        return atan2(y, x);
    }

    // 向量单位化（归一化）
```

```
    Vector unit() const
    {
        double l = this->len();
        if (sign(l) == 0)
            return (*this);
        return (*this) / l;
    }

    // 向量逆时针旋转给定弧度 angle
    Vector rotate(double angle) const
    {
        double c = cos(angle), s = sin(angle);
        return Point(x * c - y * s, x * s + y * c);
    }
};

// 计算两个向量的点积（内积）
T dot(const Vector &a, const Vector &b)
{
    return a.x * b.x + a.y * b.y;
}

// 计算两个向量的叉积（外积）
T cross(const Vector &a, const Vector &b)
{
    return a.x * b.y - b.x * a.y;
}

// 计算两个向量的夹角，范围在 [0, pi] 之间
double get_angle(const Vector &a, const Vector &b)
{
    return fabs(atan2(cross(a, b), dot(a, b)));
}

// 计算以点 a 为起点，ab 和 ac 构成的平行四边形的有向面积
T area(const Point &a, const Point &b, const Point &c)
{
    return cross(b - a, c - a);
}

// 判断三点是否共线
bool is_collinear(const Point &a, const Point &b, const Point &c)
{
    return sign(cross(b - a, c - a)) == 0;
}

// 求当前点关于点 o 的对称点
Point reflect(const Point &p, const Point &o)
{
    return o * 2 - p;
}

// 精准全平面极角排序比较器（按逆时针排序，不使用 atan2，防精度误差）
struct Polarcmp
{
    // 象限判断：1->右上(含X正)，2->左上(含Y正)，3->左下(含X负)，4->右下(含Y负)
    int quad(const Point &p) const
    {
        if (p.x >= 0 && p.y >= 0)
            return 1;
        if (p.x < 0 && p.y >= 0)
            return 2;
        if (p.x < 0 && p.y < 0)
            return 3;
        return 4;
    }

    // 比较函数
    bool operator()(const Point &a, const Point &b) const
    {
        int qa = quad(a), qb = quad(b);
        if (qa != qb)
            return qa < qb; // 象限不同，象限小的排前面
        T c = cross(a, b);
        if (sign(c) != 0)
            return sign(c) > 0;     // 象限相同，叉积大于0说明 b 在 a 的左侧（逆时针）
        return a.len2() < b.len2(); // 极角相同时，距离近的排前面
    }
};

// ========================================
// 直线
// ========================================
struct Line
{
    Point p;  // 直线上的基准点
    Vector v; // 直线的方向向量（参数方程：p + t*v）
    Line() {}
    Line(Point p, Vector v) : p(p), v(v) {}

    // 判定点 a 与有向直线（p -> p+v） Orient/To-Left 的位置关系
    // 返回值：1 在直线左侧 | -1 在直线右侧 | 0 在直线上
    int to_left(const Point &a) const
    {
```

```
        return sign(cross(v, a - p));
    }

    // 计算点 a 到该直线的垂直距离
    double dist(const Point &a) const
    {
        return fabs(cross(v, a - p)) / v.len();
    }

    // 求点 a 在该直线上的垂直投影点
    Point proj(const Point &a) const
    {
        return p + v * (dot(a - p, v) / v.len2());
    }

    // 求点 a 关于该直线的对称点（镜像点）
    Point reflect(const Point &a) const
    {
        Point pr = proj(a);
        return pr * 2.0 - a;
    }
};
// 判定两条直线的关系
// 返回值：0 相交 | 1 平行 | 2 重合
static int line_relation(const Line &a, const Line &b)
{
    if (sign(cross(a.v, b.v)) != 0)
        return 0;
    if (sign(cross(a.v, b.p - a.p)) == 0)
        return 2;
    return 1;
}

// 求两条直线 a 和 b 的唯一交点（需提前断定它们不平行）
Point inter_l2l(const Line &a, const Line &b)
{
    assert(sign(cross(a.v, b.v)) != 0);
    Vector u = a.p - b.p;
    double t = cross(b.v, u) / cross(a.v, b.v);
    return a.p + a.v * t;
}

// ========================================
// 线段
// ========================================
struct Segment
{
    Point a, b; // 线段的两个端点
    Segment() {}
    Segment(Point a, Point b) : a(a), b(b) {}

    // 判断点 p 与当前线段的位置关系：-1 在端点上 | 1 严格在线段内部 | 0 不在线段上
    int on_segment(const Point &p) const
    {
        if (p == a || p == b)
            return -1;
        if (sign(cross(p - a, p - b)) == 0 && sign(dot(p - a, p - b)) <= 0)
            return 1;
        return 0;
    }

    // 判断两线段是否相交：-1 端点处接触相交 | 1 严格在内部交叉相交 | 0 互不相交
    int is_inter(const Segment &s) const
    {
        if (on_segment(s.a) != 0 || on_segment(s.b) != 0 || s.on_segment(a) != 0 || s.on_segment(b) != 0)
            return -1;
        T c1 = cross(b - a, s.a - a), c2 = cross(b - a, s.b - a);
        T c3 = cross(s.b - s.a, a - s.a), c4 = cross(s.b - s.a, b - s.a);
        return (sign(c1) * sign(c2) < 0 && sign(c3) * sign(c4) < 0) ? 1 : 0;
    }

    // 计算点 p 到当前线段的最短距离
    double dist(const Point &p) const
    {
        if (a == b)
            return (p - a).len();
        Vector v1 = b - a, v2 = p - a, v3 = p - b;
        if (sign(dot(v1, v2)) < 0)
            return v2.len();
        if (sign(dot(v1, v3)) > 0)
            return v3.len();
        Line l{a, b - a};
        return l.dist(p); // 正常向线段作垂线距离
    }

    // 计算当前线段到另一条线段 s 的最短距离
    double dist(const Segment &s) const
    {
        if (is_inter(s) != 0)
            return 0;
        return min({dist(s.a), dist(s.b), s.dist(a), s.dist(b)});
    }
};
```

```
// 求两条线段的交点坐标（调用前必须先确保 is_inter 返回非 0 状态）
Point inter_s2s(const Segment &s1, const Segment &s2)
{
    assert(s1.is_inter(s2) == 0);
    Line l1{s1.a, s1.b - s1.a};
    Line l2{s2.a, s2.b - s2.a};
    return inter_l2l(l1, l2);
}

// ========================================
// 圆
// ========================================
struct Circle
{
    Point o;
    T r;
    Circle() {};
    Circle(Point o, T r) : o(o), r(r) {};

    bool operator==(const Circle &a) const
    {
        return o == a.o && cmp(r, a.r) == 0;
    }
};

// ========================================
// 多边形
// ========================================
struct Polygon
{
    vector<Point> p; // 顶点按边界顺序存储，顺/逆时针均可
    Polygon() {}
    Polygon(const vector<Point> &p) : p(p) {}

    int size() const
    {
        return p.size();
    }

    // 有向面积的两倍：逆时针为正，顺时针为负
    T area2() const
    {
        T res = 0;
        for (int i = 0; i < size(); i++)
            res += cross(p[i], p[(i + 1) % size()]);
        return res;
    }

    // 多边形面积
    double area() const
    {
        return fabs(area2()) / 2;
    }

    // 多边形周长
    double perimeter() const
    {
        double res = 0;
        for (int i = 0; i < size(); i++)
            res += (p[i] - p[(i + 1) % size()]).len();
        return res;
    }

    // 判断点 a 与多边形的位置关系：1 内部 | -1 边界上 | 0 外部
    int contain(const Point &a) const
    {
        bool in = false;
        for (int i = 0; i < size(); i++)
        {
            Point u = p[i], v = p[(i + 1) % size()];
            if (Segment(u, v).on_segment(a) != 0)
                return -1;
            if (sign(u.y - a.y) != sign(v.y - a.y) &&
                sign(cross(v - u, a - u)) == (sign(v.y - a.y) > 0 ? 1 : -1))
                in = !in;
        }
        return in ? 1 : 0;
    }

    // 判断是否为凸多边形，退化情形返回 false
    bool is_convex() const
    {
        if (size() < 3 || sign(area2()) == 0)
            return false;
        int pre = 0;
        for (int i = 0; i < size(); i++)
        {
            int cur = sign(cross(p[(i + 1) % size()] - p[i], p[(i + 2) % size()] - p[(i + 1) % size()]));
            if (cur == 0)
                continue;
            if (pre != 0 && cur != pre)
                return false;
            pre = cur;
        }
```

```
        return pre != 0;
    }
};

// ========================================
// 凸包
// ========================================
struct Convex : Polygon
{
    Convex() {}
    Convex(vector<Point> q, bool keep_collinear = false)
    {
        build(q, keep_collinear);
    }

    // Andrew 单调链：默认删除边上的共线点；keep_collinear 为 true 时保留
    void build(vector<Point> q, bool keep_collinear = false)
    {
        sort(q.begin(), q.end());
        q.erase(unique(q.begin(), q.end()), q.end());
        p.clear();
        if (q.size() <= 1)
        {
            p = q;
            return;
        }
        bool collinear = true;
        for (int i = 2; i < (int)q.size(); i++)
            if (sign(cross(q[1] - q[0], q[i] - q[0])) != 0)
            {
                collinear = false;
                break;
            }
        if (collinear)
        {
            p = keep_collinear ? q : vector<Point>{q.front(), q.back()};
            return;
        }

        auto check = [&](const Point &a, const Point &b, const Point &c)
        {
            int s = sign(cross(b - a, c - b));
            return keep_collinear ? s < 0 : s <= 0;
        };

        for (const Point &a : q)
        {
            while (p.size() >= 2 && check(p[p.size() - 2], p.back(), a))
                p.pop_back();
            p.push_back(a);
        }
        int lower_size = p.size();
        for (int i = (int)q.size() - 2; i >= 0; i--)
        {
            while ((int)p.size() > lower_size && check(p[p.size() - 2], p.back(), q[i]))
                p.pop_back();
            p.push_back(q[i]);
        }
        p.pop_back(); // 不重复存储起点
    }
    // 旋转卡壳求凸包直径
    double diameter()
    {
        int n = p.size();
        if (n <= 1)
            return 0;
        if (n == 2)
            return (p[0] - p[1]).len();
        double ans = 0;
        for (int i = 0, j = i + 1; i < n; i++)
        {
            auto u = p[i], v = p[(i + 1) % n];
            while (cmp(abs(cross(v - u, p[(j + 1) % n] - u)), abs(cross(v - u, p[j] - u))) == 1)
            {
                j = (j + 1) % n;
            }
            ans = max(ans, (v - p[j]).len());
            ans = max(ans, (u - p[j]).len());
        }
        return ans;
    }
};

// ========================================
// 半平面交
// 左侧为合法区域
// ========================================

// 去除同方向半平面
vector<Line> uniquel(vector<Line> &l)
{
    vector<Line> ll;
    for (auto line : l)
    {
```

```
        if (ll.empty() || sign(cross(ll.back().v, line.v)) != 0)
            ll.push_back(line);
        else
        {
            if (ll.back().to_left(line.p) == 1)
            {
                ll.back() = line;
            }
        }
    }
    return ll;
}

Convex inter_con(vector<Line> &line)
{
    ranges::sort(line, [](auto &a, auto &b)
                 { return Polarcmp()(a.v, b.v); });
    line = uniquel(line);
    deque<Line> dq;
    for (auto &l : line)
    {
        while (dq.size() > 1)
        {
            Point p = inter_l2l(dq[dq.size() - 2], dq.back());
            // 0: 边界, 1: 左侧
            if (l.to_left(p) >= 0)
                break;
            dq.pop_back();
        }
        while (dq.size() > 1)
        {
            Point p = inter_l2l(dq[0], dq[1]);

            if (l.to_left(p) >= 0)
                break;

            dq.pop_front();
        }
        dq.push_back(l);
    }
    while (dq.size() > 2)
    {
        Point p = inter_l2l(dq[dq.size() - 2], dq.back());
        if (dq[0].to_left(p) >= 0)
            break;
        dq.pop_back();
    }
    while (dq.size() > 2)
    {
        Point p = inter_l2l(dq[0], dq[1]);
        if (dq.back().to_left(p) >= 0)
            break;
        dq.pop_front();
    }
    vector<Point> res;
    if (dq.size() < 3)
        return res;
    int n = dq.size();
    for (int i = 0; i < n; i++)
    {
        res.push_back(inter_l2l(dq[i], dq[(i + 1) % n]));
    }
    return Convex(res);
}

// ========================================
// 闵可夫斯基和
// 输入：两个逆时针凸多边形
// 返回：A+B 的凸多边形
// ========================================

Convex minkowski(Convex &a, Convex &b)
{
    int n = a.size(), m = b.size();

    // 找最低点
    auto get = [](vector<Point> &p)
    {
        int pos = 0;

        for (int i = 1; i < p.size(); i++)
        {
            if (p[i].y < p[pos].y ||
                (cmp(p[i].y, p[pos].y) == 0 &&
                 p[i].x < p[pos].x))
            {
                pos = i;
            }
        }

        rotate(p.begin(), p.begin() + pos, p.end());
    };
    auto &pa = a.p, &pb = b.p;
    get(pa), get(pb);
```

```
    vector<Vector> va(n), vb(m);
    for (int i = 0; i < n; i++)
    {
        va[i] = pa[(i + 1) % n] - pa[i];
    }

    for (int i = 0; i < m; i++)
    {
        vb[i] = pb[(i + 1) % m] - pb[i];
    }

    vector<Point> res;

    int i = 0, j = 0;

    Point cur = pa[0] + pb[0];

    while (i != n || j != m)
    {
        Vector v;
        if (i == n)
            v = vb[j++];
        else if (j == m)
            v = va[i++];
        else
        {
            int c = sign(cross(va[i], vb[j]));

            if (c > 0)
            {
                v = va[i++];
            }
            else if (c < 0)
            {
                v = vb[j++];
            }
            else
            {
                v = va[i++] + vb[j++];
            }
        }
        cur = cur + v;
        res.push_back(cur);
    }
    res.pop_back();
    return Convex(res);
}
```